

Java Complete Guide + DSA Mapping

> Covers everything: Core Language, OOP, Collections, Advanced Concepts, and a full DSA mapping — with practical examples and interview-ready insights. Assumes **zero** prior Java knowledge.

Table of Contents

Part 1 — Complete Java Notes

1. [What is Java?](#1-what-is-java)
2. [Setting Up Java](#2-setting-up-java)
3. [Your First Java Program](#3-your-first-java-program)
4. [Variables & Data Types](#4-variables--data-types)
5. [Type Casting](#5-type-casting)
6. [Operators](#6-operators)
7. [Control Flow](#7-control-flow)
8. [Loops](#8-loops)
9. [Arrays](#9-arrays)
10. [Strings](#10-strings)
11. [Methods (Functions)](#11-methods-functions)
12. [Object-Oriented Programming — Classes & Objects](#12-object-oriented-programming--classes--objects)
13. [Constructors](#13-constructors)
14. [Encapsulation — Access Modifiers & Getters/Setters](#14-encapsulation--access-modifiers--getterssetters)
15. [Inheritance](#15-inheritance)
16. [Polymorphism](#16-polymorphism)
17. [Abstraction — Abstract Classes & Interfaces](#17-abstraction--abstract-classes--interfaces)
18. [Static Keyword](#18-static-keyword)
19. [final Keyword](#19-final-keyword)
20. [Exception Handling](#20-exception-handling)
21. [Generics](#21-generics)
22. [Collections Framework](#22-collections-framework)
23. [ArrayList](#23-arraylist)
24. [LinkedList](#24-linkedlist)
25. [Stack](#25-stack)
26. [Queue & Deque](#26-queue--deque)
27. [HashMap & HashSet](#27-hashmap--hashset)
28. [TreeMap & TreeSet](#28-treemap--treeset)
29. [PriorityQueue (Heap)](#29-priorityqueue-heap)
30. [Sorting & Comparators](#30-sorting--comparators)

- 31. [Recursion](#31-recursion)
- 32. [Lambda Expressions & Functional Interfaces](#32-lambda-expressions--functional-interfaces)
- 33. [Streams API](#33-streams-api)
- 34. [StringBuilder](#34-stringbuilder)

Part 2 — Java for DSA

- 35. [Java Needed for DSA](#part-2--java-needed-for-dsa)

Part 1 — Complete Java Notes

1. What is Java?

Explanation

Java is a **high-level, compiled, statically typed, object-oriented** programming language. It runs on the **Java Virtual Machine (JVM)**, which means Java code compiles once and runs anywhere — on Windows, Mac, Linux, or any device with a JVM.

Key characteristics:

- **Compiled** → Java source code (`.java`) compiles to **bytecode** (`.class`), then the JVM runs the bytecode
- **Statically typed** → every variable's type is declared at compile time and cannot change
- **Object-Oriented** → almost everything is organized into classes and objects
- **Strongly typed** → no implicit type coercion like JavaScript; types must match or be explicitly cast
- **Platform-independent** → "Write Once, Run Anywhere" (WORA) — the JVM handles platform differences
- **Garbage collected** → memory management is automatic (JVM cleans up unused objects)

How Java code runs:

...

YourCode.java → [Java Compiler: javac] → YourCode.class (bytecode) → [JVM] → Runs!

...

Java vs JavaScript — key differences:

Feature	Java	JavaScript
Typing	Static (type declared upfront)	Dynamic (type at runtime)

Compilation	Compiled to bytecode	Interpreted / JIT compiled
OOP	Strictly class-based	Prototype-based
Threading	True multi-threading	Single-threaded + event loop
Main use	Backend, Android, DSA	Web frontend, backend (Node.js)
Type errors	Caught at compile time	Only caught at runtime

Examples

```
```java
// Basic Java program structure
public class HelloWorld {
 public static void main(String[] args) {
 System.out.println("Hello, World!"); // → Hello, World!
 }
}
```
```

Key Points

- Every Java program must have **at least one class**
- The program starts executing from the **main** method — this is mandatory
- Java is **case-sensitive** → `MyClass` ≠ `myclass`
- Every statement ends with a **semicolon** `;`
- File name must match the class name exactly: `HelloWorld.java` contains `class HelloWorld`

Common Mistakes

```
```java
// Mistake 1 — missing semicolon
int x = 5 // ❌ compile error
int x = 5; // ✅

// Mistake 2 — file name not matching class name
// File: hello.java, but class is HelloWorld → compile error
// File MUST be: HelloWorld.java

// Mistake 3 — wrong main signature
public void main() {} // ❌ JVM won't find entry point
public static void main(String[] args) {} // ✅
```
```

Extra Notes

- Java was created by James Gosling at Sun Microsystems in 1995
- Now owned and maintained by Oracle; OpenJDK is the open-source version
- LTS (Long-Term Support) versions: Java 8, 11, 17, 21 — prefer Java 17+ for new projects
- For DSA (LeetCode, interviews) — Java is one of the top choices because of its rich Collections framework

2. Setting Up Java

Explanation

To write and run Java code, you need two things:

1. **JDK (Java Development Kit)** — includes the compiler (`javac`) + runtime (JVM)
2. **An IDE or text editor** — IntelliJ IDEA (best for Java), VS Code, or just terminal

Setup Steps

Step 1 — Install JDK

- Download from: <https://adoptium.net> (free, open-source — Eclipse Temurin)
- Or use: <https://jdk.java.net> (official OpenJDK)
- Install and verify:

```
```bash
java -version # should show java version "17..." or similar
javac -version # should show javac 17...
```
```

Step 2 — Compile & Run (Terminal)

```
```bash
Create file: Hello.java
Compile:
javac Hello.java # creates Hello.class

Run:
java Hello # runs the compiled bytecode
```
```

Step 3 — IDE Setup (Recommended: IntelliJ IDEA)

- Download IntelliJ IDEA Community Edition (free): <https://www.jetbrains.com/idea/>
- Create New Project → Java → Select JDK → Done
- IntelliJ compiles and runs automatically — no manual `javac` needed

For LeetCode / Online Judges:

- No setup needed — just select Java as your language
- Your code runs inside a class — LeetCode provides the class skeleton, you fill in the method

Key Points

- **JDK** = Compiler + JVM + Libraries (you need this to develop)
- **JRE** = JVM + Libraries only (just to run; not develop — older distinction, modern JDK includes both)
- For DSA practice, online IDEs work fine: <https://www.jdoodle.com>, <https://repl.it>, LeetCode itself

3. Your First Java Program

Explanation

Every Java program follows a specific structure. Let's break it down completely.

Examples

```
```java
```

```
// File: HelloWorld.java
```

```
public class HelloWorld { // class declaration — must match filename
```

```
 public static void main(String[] args) { // entry point — JVM starts here
```

```
 // Printing to console
```

```
 System.out.println("Hello, World!"); // prints + newline
```

```
 System.out.print("No newline here"); // prints without newline
```

```
 System.out.println(); // just a newline
```

```
 // Printing variables
```

```
 int age = 25;
```

```
 String name = "Alice";
```

```
 System.out.println("Name: " + name + ", Age: " + age); // string concatenation
```

```
 System.out.printf("Name: %s, Age: %d%n", name, age); // formatted output
```

```
 // Reading input (from keyboard)
```

```
 java.util.Scanner scanner = new java.util.Scanner(System.in);
```

```
 System.out.print("Enter your name: ");
```

```
 String input = scanner.nextLine();
```

```
 System.out.println("Hello, " + input + "!");
```

```
 scanner.close();
```

```
}
}
...
```

**\*\*Anatomy of every line:\*\***

```
```java  
public class HelloWorld {  
    // ^   ^   ^  
    // access keyword  class name (must match filename)  
    // modifier  
  
    public static void main(String[] args) {  
        // ^   ^   ^   ^   ^  
        // access static return  method  parameter (command line args)  
        // mod.  method type  name  — array of Strings  
    }  
}
```

Key Points

- `public` → accessible from anywhere
- `static` → belongs to the class, not an instance — JVM calls main without creating an object
- `void` → this method returns nothing
- `main` → the name the JVM looks for to start execution
- `String[] args` → command line arguments (can be ignored in basic programs)
- `System.out.println()` → `System` is a class, `out` is a static field (PrintStream), `println` is a method

Common Mistakes

```
```java  
// Mistake 1 — System.out.println with wrong case
system.out.println("Hi"); // ❌ — 'system' is lowercase
System.out.println("Hi"); // ✅
```

// Mistake 2 — Forgetting to close brackets

```
public class Hello {
 public static void main(String[] args) {
 System.out.println("Hi");
 }
// ❌ — missing closing brace for class
```

// Mistake 3 — Using Scanner without importing

```
Scanner sc = new Scanner(System.in); // ❌ — Scanner not found
```

```
import java.util.Scanner; // ✓ — add this at top of file

Scanner sc = new Scanner(System.in); // ✓

...

```

## ## 4. Variables & Data Types

### ### Explanation

Java is **statically typed** — every variable must have its type declared when created, and that type cannot change. Java has two categories of types:

**Primitive Types** (8 total — stored directly in memory, not objects):

Type	Size	Range	Default	Example
	----	-----	-----	-----
`byte`	1 byte	-128 to 127	0	`byte b = 10;`
`short`	2 bytes	-32,768 to 32,767	0	`short s = 1000;`
`int`	4 bytes	~2.1B to 2.1B	0	`int x = 42;`
`long`	8 bytes	~9.2 quintillion	0L	`long l = 100L;`
`float`	4 bytes	~7 decimal digits	0.0f	`float f = 3.14f;`
`double`	8 bytes	~15 decimal digits	0.0	`double d = 3.14;`
`char`	2 bytes	0 to 65,535 (Unicode)	`\u0000`	`char c = 'A';`
`boolean`	1 bit	true / false	false	`boolean flag = true;`

**Reference Types** (stored as references/pointers to objects in heap memory):

- `String` → text
- Arrays → `int[]`, `String[]`, etc.
- Classes → `ArrayList`, `HashMap`, your custom classes, etc.
- The value stored is a **reference** (memory address), not the actual data

### ### Examples

```
```java
public class DataTypes {
    public static void main(String[] args) {

        // === PRIMITIVES ===
        int age = 25;
        long population = 8_000_000_000L; // L suffix for long; _ for readability
        double price = 19.99;
        float weight = 72.5f;           // f suffix for float
    }
}
```

```
char grade = 'A';           // single quotes for char
```

```
boolean isStudent = true;
```

```
// === REFERENCE TYPES ===
```

```
String name = "Alice";      // double quotes for String
```

```
int[] numbers = {1, 2, 3, 4, 5}; // array
```

```
// === TYPE INFERENCE WITH var (Java 10+) ===
```

```
var city = "Paris"; // compiler infers type as String
```

```
var count = 42; // compiler infers type as int
```

```
// var can be used in local variables only (inside methods)
```

```
// === CONSTANTS — use 'final' ===
```

```
final int MAX_SIZE = 100;
```

```
final double PI = 3.14159;
```

```
// MAX_SIZE = 200; // ❌ compile error — cannot reassign final
```

```
// === DEFAULT VALUES (for class fields, not local variables) ===
```

```
// int → 0, double → 0.0, boolean → false, String → null
```

```
// Local variables MUST be initialized before use — no defaults!
```

```
// === PRINTING ===
```

```
System.out.println("Age: " + age);
```

```
System.out.println("Name: " + name);
```

```
System.out.println("Grade: " + grade);
```

```
System.out.printf("Price: %.2f%n", price); // %.2f = 2 decimal places
```

```
// === COMPARING PRIMITIVES vs REFERENCES ===
```

```
int a = 5, b = 5;
```

```
System.out.println(a == b); // true — primitives compared by VALUE
```

```
String s1 = new String("hello");
```

```
String s2 = new String("hello");
```

```
System.out.println(s1 == s2); // false — different objects in memory!
```

```
System.out.println(s1.equals(s2)); // true — compare content with .equals()
```

```
// String literals are cached (string pool) — this works:
```

```
String s3 = "hello";
```

```
String s4 = "hello";
```

```
System.out.println(s3 == s4); // true — both point to same pool object
```

```
}
```

```
}
```

```
...
```

Key Points

- ****Always use `==` for primitives, `.equals()` for objects (especially Strings)****
- `int` is the default choice for whole numbers; `double` for decimals
- `long` needs `L` suffix: `100L`; `float` needs `f` suffix: `3.14f`
- `char` uses single quotes `A`; `String` uses double quotes `"hello"`
- `final` is Java's equivalent of JavaScript's `const` — the binding cannot be reassigned
- Local variables have NO default values — must initialize before use (compiler enforces this)

Common Mistakes

```
```java
```

```
// Mistake 1 — comparing Strings with ==
```

```
String s1 = new String("hello");
```

```
String s2 = new String("hello");
```

```
if (s1 == s2) { } // ❌ — compares references, almost always false
```

```
if (s1.equals(s2)) { } // ✅ — compares content
```

```
// Mistake 2 — integer overflow (silent!)
```

```
int max = Integer.MAX_VALUE; // 2147483647
```

```
int overflow = max + 1; // -2147483648 — wraps around! No error thrown
```

```
long safe = (long) max + 1; // ✅ — cast to long first
```

```
// Mistake 3 — float/double precision
```

```
double result = 0.1 + 0.2;
```

```
System.out.println(result); // 0.30000000000000004 — floating point imprecision
```

```
// For exact decimals, use BigDecimal
```

```
// Mistake 4 — forgetting L for large long literals
```

```
long big = 10000000000; // ❌ compile error — literal exceeds int range
```

```
long big = 10000000000L; // ✅
```

```
// Mistake 5 — using uninitialized local variable
```

```
int x;
```

```
System.out.println(x); // ❌ compile error: variable x might not have been initialized
```

```
...
```

### ### Extra Notes

- `Integer.MAX\_VALUE` = 2,147,483,647 (~2.1 billion) — critical for DSA overflow checks
- `Long.MAX\_VALUE` = 9,223,372,036,854,775,807 — use when int might overflow

- **Wrapper classes**: each primitive has an object version: `int` → `Integer`, `double` → `Double`, `char` → `Character`, `boolean` → `Boolean` — needed when using Collections (which require objects, not primitives)

- **Autoboxing/Unboxing**: Java automatically converts between `int` and `Integer` when needed

---

## ## 5. Type Casting

### ### Explanation

Since Java is statically typed, converting between types requires **casting**. Two kinds:

- **Widening (implicit)** → smaller type → larger type. No data loss, Java does it automatically.

- **Narrowing (explicit)** → larger type → smaller type. May lose data; you must cast manually.

...

`byte` → `short` → `int` → `long` → `float` → `double` (widening direction →)

...

### ### Examples

```
```java
```

```
public class TypeCasting {
    public static void main(String[] args) {

        // === WIDENING (automatic — safe) ===
        int i = 100;
        long l = i;    // int → long — automatic
        double d = i;  // int → double — automatic
        System.out.println(d); // 100.0

        // === NARROWING (manual cast — may lose data) ===
        double pi = 3.99;
        int piInt = (int) pi; // (int) is the cast operator
        System.out.println(piInt); // 3 — decimal part is TRUNCATED (not rounded!)

        long bigNum = 1234567890123L;
        int smallNum = (int) bigNum; // data loss! only lower 32 bits kept
        System.out.println(smallNum); // unexpected value

        // === char ↔ int CASTING (very useful for DSA!) ===
        char ch = 'A';
```

```
int ascii = (int) ch;    // char → int: gets ASCII/Unicode value
```

```
System.out.println(ascii); // 65
```

```
char back = (char) 66;   // int → char
```

```
System.out.println(back); // B
```

```
// DSA pattern: get index of letter (a=0, b=1, c=2...)
```

```
char letter = 'e';
```

```
int index = letter - 'a'; // 'e' - 'a' = 101 - 97 = 4
```

```
System.out.println(index); // 4
```

```
// === String ↔ int conversions ===
```

```
// String to int:
```

```
String numStr = "42";
```

```
int parsed = Integer.parseInt(numStr); // "42" → 42
```

```
double parsedD = Double.parseDouble("3.14"); // "3.14" → 3.14
```

```
// int to String:
```

```
int num = 42;
```

```
String s1 = String.valueOf(num); // 42 → "42"
```

```
String s2 = Integer.toString(num); // 42 → "42"
```

```
String s3 = "" + num;           // concatenation trick: "" + 42 = "42"
```

```
// === Comparing types after casting ===
```

```
System.out.println(5 / 2);      // 2 — INTEGER DIVISION (both operands are int)
```

```
System.out.println(5.0 / 2);    // 2.5 — at least one is double
```

```
System.out.println((double) 5 / 2); // 2.5 — cast one operand to double
```

```
}
```

```
}
```

```
...
```

Key Points

- **Integer division truncates**: `7 / 2 = 3`, NOT `3.5`. This is one of the most common DSA bugs!
- To get a decimal result from int division, cast one to `double`: `(double) a / b`
- `char - 'a'` gives the 0-based index of a lowercase letter (DSA essential)
- `Integer.parseInt()` throws `NumberFormatException` if the string isn't a valid number

Common Mistakes

```
```java
```

```
// Mistake 1 — integer division when you need decimal
```

```
int a = 7, b = 2;
```

double result = a / b; // ❌ — 3.0 (division happens first as int!)

double result = (double) a / b; // ✅ — 3.5

// Mistake 2 — cast truncates, doesn't round

double x = 3.9;

int xi = (int) x; // 3, NOT 4

// Use Math.round(x) if you want rounding

// Mistake 3 — char arithmetic forgetting it uses Unicode values

char c = 'A' + 1; // ❌ compile error — 'A'+1 is int, can't assign to char without cast

char c = (char)('A' + 1); // ✅ → 'B'

...

---

## ## 6. Operators

### ### Explanation

Operators in Java are similar to most languages. Java does NOT have JavaScript's `===`, `??`, `?.` — but it has powerful bitwise and arithmetic operators.

### ### Examples

```java

public class Operators {

public static void main(String[] args) {

// === ARITHMETIC ===

int x = 10, y = 3;

System.out.println(x + y); // 13

System.out.println(x - y); // 7

System.out.println(x * y); // 30

System.out.println(x / y); // 3 — INTEGER DIVISION!

System.out.println(x % y); // 1 — modulo (remainder)

// No ** in Java — use Math.pow()

System.out.println(Math.pow(2, 10)); // 1024.0

// Pre/post increment

int a = 5;

System.out.println(a++); // 5 — returns then increments

System.out.println(a); // 6

```
System.out.println(++a); // 7 – increments then returns
```

```
// === ASSIGNMENT ===
```

```
x += 5; // x = x + 5
```

```
x -= 3; // x = x - 3
```

```
x *= 2; // x = x * 2
```

```
x /= 4; // x = x / 4
```

```
x %= 3; // x = x % 3
```

```
// === COMPARISON (always returns boolean) ===
```

```
System.out.println(5 > 3); // true
```

```
System.out.println(5 >= 5); // true
```

```
System.out.println(5 < 3); // false
```

```
System.out.println(5 == 5); // true – for primitives only!
```

```
System.out.println(5 != 3); // true
```

```
// === LOGICAL ===
```

```
boolean p = true, q = false;
```

```
System.out.println(p && q); // false – AND
```

```
System.out.println(p || q); // true – OR
```

```
System.out.println(!p); // false – NOT
```

```
// Short-circuit: && stops if first is false; || stops if first is true
```

```
int n = 0;
```

```
if (n != 0 && 10 / n > 1) { } // safe – n != 0 is false → stops, no division by zero
```

```
// === TERNARY ===
```

```
int score = 85;
```

```
String grade = score >= 90 ? "A" : score >= 80 ? "B" : "C";
```

```
System.out.println(grade); // B
```

```
// === BITWISE (critical for DSA!) ===
```

```
int bits = 0b1010; // binary literal = 10
```

```
System.out.println(5 & 3); // 1 – AND: 101 & 011 = 001
```

```
System.out.println(5 | 3); // 7 – OR: 101 | 011 = 111
```

```
System.out.println(5 ^ 3); // 6 – XOR: 101 ^ 011 = 110
```

```
System.out.println(~5); // -6 – NOT: flips all bits
```

```
System.out.println(5 << 1); // 10 – left shift (multiply by 2)
```

```
System.out.println(5 >> 1); // 2 – right shift (divide by 2, preserves sign)
```

```
System.out.println(-5 >>> 1); // unsigned right shift (fills with 0)
```

```
// DSA bitwise tricks:
```

```

System.out.println(6 & 1); // 0 – even number check
System.out.println(7 & 1); // 1 – odd number check
System.out.println(8 >> 1); // 4 – fast divide by 2
int mid = 2 + ((8 - 2) >> 1); // safe midpoint without overflow

// === instanceof ===
String str = "hello";
System.out.println(str instanceof String); // true
System.out.println(str instanceof Object); // true – everything extends Object
}
}
...

```

Key Points

- Java has NO `**` operator – use `Math.pow(base, exp)` which returns `double`
- `==` compares `**reference**` for objects – use `.equals()` for content equality
- `%` with negative numbers: `-7 % 3 = -1` in Java (sign follows dividend)
- Bitwise operators are extremely useful in DSA for optimization
- `>>>` (unsigned right shift) fills with 0 regardless of sign – useful for safe midpoint

Common Mistakes

```

```java
// Mistake 1 – integer division
double avg = (1 + 2 + 3) / 3; // ❌ → 2.0 (division of ints first)
double avg = (1 + 2 + 3) / 3.0; // ✅ → 2.0 (correct: 2.0)
double avg = (double)(1 + 2 + 3) / 3; // ✅

// Mistake 2 – using == for String comparison
String s1 = "hello", s2 = "hello";
if (s1 == s2) {} // may work for literals (string pool), but unreliable
if (s1.equals(s2)) {} // ✅ always correct

```

```

// Mistake 3 – negative modulo
int result = -7 % 3; // -1 in Java (not 2 as in math)
// Fix: ((n % m) + m) % m – always positive
int positive = ((-7 % 3) + 3) % 3; // 2
...

```

## ## 7. Control Flow

### ### Explanation

Control flow in Java is very similar to JavaScript — `if/else`, `switch`, but with some differences: Java's `switch` can use `String`, `int`, `char`, and `enum` types.

### ### Examples

```
```java
public class ControlFlow {
    public static void main(String[] args) {

        // === if / else if / else ===
        int score = 85;

        if (score >= 90) {
            System.out.println("A");
        } else if (score >= 80) {
            System.out.println("B");
        } else if (score >= 70) {
            System.out.println("C");
        } else {
            System.out.println("F");
        }
        // → B

        // === switch statement ===
        String day = "Monday";

        switch (day) {
            case "Monday":
            case "Tuesday":
            case "Wednesday":
            case "Thursday":
            case "Friday":
                System.out.println("Weekday");
                break;          // MUST have break — or it falls through!
            case "Saturday":
            case "Sunday":
                System.out.println("Weekend");
                break;
            default:
                System.out.println("Unknown");
        }
    }
}
```

```

}

// === switch expression (Java 14+) – cleaner! ===
String type = switch (day) {
    case "Saturday", "Sunday" -> "Weekend";
    default -> "Weekday";
};
System.out.println(type); // Weekday

// === switch with int ===
int num = 2;
switch (num) {
    case 1: System.out.println("One"); break;
    case 2: System.out.println("Two"); break;
    case 3: System.out.println("Three"); break;
    default: System.out.println("Other");
}

// === guard clauses (early return pattern) ===
// BAD – deeply nested
// GOOD – guard clauses
}

// Guard clause example
static String processOrder(String order, int[] items) {
    if (order == null) return "No order";
    if (items == null) return "No items";
    if (items.length == 0) return "Empty order";
    return "Processing " + items.length + " items";
}
}
...

```

Key Points

- Java `switch` supports: `int`, `byte`, `short`, `char`, `String`, `enum`
- Always include `break` in switch cases – omitting causes **fall-through** (execution continues to next case)
- Java 14+ switch expressions with `->` are cleaner and don't need `break`
- Java does NOT have `??` (nullish coalescing) or `?.` (optional chaining) like JavaScript

Common Mistakes

```
```java
```

// Mistake 1 – missing break (unintentional fall-through)

```
switch (x) {
 case 1:
 System.out.println("One"); // no break!
 case 2:
 System.out.println("Two"); // ALSO runs when x == 1!
 case 3:
 System.out.println("Three");
 break;
}
```

// Mistake 2 – switch on null (throws NullPointerException)

```
String s = null;
switch (s) { ... } // ❌ NullPointerException
```

// Mistake 3 – comparing with == in if condition by accident

```
int a = 5;
if (a = 10) {} // ❌ compile error in Java (unlike C/C++) – assignment in if not allowed
if (a == 10) {} // ✅
...

```

## ## 8. Loops

### ### Explanation

Java has the same loop types as most C-style languages. No `for...of` or `for...in` like JavaScript, but it has the enhanced `for-each` loop which works on arrays and collections.

### ### Examples

```
```java
```

```
import java.util.List;  
import java.util.Arrays;
```

```
public class Loops {  
    public static void main(String[] args) {
```

```
        // === Classic for loop ===
```

```
        for (int i = 0; i < 5; i++) {
```

```
            System.out.print(i + " "); // 0 1 2 3 4
```

```

}

// Reverse
for (int i = 4; i >= 0; i--) {
    System.out.print(i + " "); // 4 3 2 1 0
}

// === while loop ===
int count = 0;
while (count < 3) {
    System.out.println(count); // 0 1 2
    count++;
}

// === do-while (runs at least once) ===
int x = 10;
do {
    System.out.println(x); // 10 — runs once even though 10 < 5 is false
    x++;
} while (x < 5);

// === for-each (enhanced for loop) ===
// Works on arrays AND any Collection (List, Set, etc.)
int[] numbers = {10, 20, 30, 40, 50};
for (int num : numbers) { // read as "for each num in numbers"
    System.out.print(num + " "); // 10 20 30 40 50
}

String[] fruits = {"apple", "banana", "cherry"};
for (String fruit : fruits) {
    System.out.println(fruit); // apple, banana, cherry
}

// ❌ for-each doesn't give index — use classic for if you need index
// ❌ for-each can't modify array values (modifying 'num' doesn't change array)

// === break and continue ===
for (int i = 0; i < 10; i++) {
    if (i == 3) continue; // skip 3
    if (i == 7) break; // stop at 7
    System.out.print(i + " "); // 0 1 2 4 5 6
}

```

// === labeled break (for nested loops — very useful in DSA!) ===

outer:

```
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 3; j++) {  
        if (i == 1 && j == 1) break outer; // exits BOTH loops  
        System.out.println(i + "," + j);  
    }  
}
```

// prints: 0,0 0,1 0,2 1,0 (stops at 1,1)

// === Iterating with index over List ===

```
List<String> list = Arrays.asList("a", "b", "c");
```

```
for (int i = 0; i < list.size(); i++) {  
    System.out.println(i + ": " + list.get(i));  
}
```

// OR forEach with lambda (Java 8+):

```
list.forEach(item -> System.out.println(item));
```

```
list.forEach(System.out::println); // method reference — same thing
```

```
}  
}  
...
```

Key Points

- `for-each` → use when you need values and don't need the index
- `classic for` → use when you need index, or need to modify elements
- `while` → use when you don't know number of iterations upfront
- `do-while` → use when you need to execute at least once (e.g., user input validation)
- Java's `for-each` works on arrays AND any `Iterable` (all Collection classes)

Common Mistakes

```
```java
```

// Mistake 1 — modifying array in for-each (doesn't work!)

```
int[] arr = {1, 2, 3};
```

```
for (int n : arr) {
 n = n * 2; // ❌ modifies local copy, NOT the array
}
```

// Fix: use classic for loop

```
for (int i = 0; i < arr.length; i++) {
```

```
 arr[i] = arr[i] * 2; // ✅
```

```
}
```

// Mistake 2 – infinite loop

```
int i = 0;
while (i < 5) {
 System.out.println(i);
 // forgot i++ – infinite loop!
}
```

// Mistake 3 – off-by-one

```
for (int i = 0; i <= arr.length; i++) { // should be i < arr.length
 System.out.println(arr[i]); // ❌ ArrayIndexOutOfBoundsException on last iteration
}
```

// Mistake 4 – ConcurrentModificationException (modifying collection while iterating)

```
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3));
for (int n : list) {
 if (n == 2) list.remove(n); // ❌ ConcurrentModificationException
}
// Fix: use Iterator or collect items to remove, then remove after loop
...

```

## ## 9. Arrays

### ### Explanation

Arrays in Java are **fixed-size**, **zero-indexed**, and **type-specific**. Once created, the size cannot change. For dynamic sizing, use `ArrayList`.

### ### Examples

```
```java
```

```
import java.util.Arrays;
```

```
public class ArraysDemo {
    public static void main(String[] args) {
```

```
        // === CREATION ===
```

```
        int[] arr1 = {1, 2, 3, 4, 5};    // array literal
```

```
        int[] arr2 = new int[5];         // size 5, all zeros
```

```
        String[] arr3 = new String[3];   // size 3, all null
```

```
        int[] arr4 = new int[]{10, 20, 30}; // explicit new
```

// 2D arrays

```
int[][] matrix = new int[3][4]; // 3 rows, 4 columns
```

```
int[][] grid = {{1,2,3},{4,5,6},{7,8,9}}; // 2D literal
```

// Jagged array (rows of different sizes)

```
int[][] jagged = new int[3][];
```

```
jagged[0] = new int[2];
```

```
jagged[1] = new int[4];
```

```
jagged[2] = new int[1];
```

// === ACCESSING ===

```
System.out.println(arr1[0]); // 1 — first element
```

```
System.out.println(arr1[4]); // 5 — last element
```

```
System.out.println(arr1.length); // 5 — length is a FIELD (not method!)
```

```
// arr1[-1] → ❌ ArrayIndexOutOfBoundsException
```

```
// arr1[5] → ❌ ArrayIndexOutOfBoundsException
```

// === MODIFYING ===

```
arr2[0] = 10;
```

```
arr2[1] = 20;
```

// === TRAVERSING ===

```
for (int i = 0; i < arr1.length; i++) {
```

```
    System.out.print(arr1[i] + " ");
```

```
}
```

// 2D traversal

```
for (int i = 0; i < grid.length; i++) {
```

```
    for (int j = 0; j < grid[i].length; j++) {
```

```
        System.out.print(grid[i][j] + " ");
```

```
    }
```

```
    System.out.println();
```

```
}
```

// === JAVA ARRAYS CLASS (utility methods) ===

```
int[] nums = {5, 3, 8, 1, 9, 2};
```

```
Arrays.sort(nums); // in-place sort ascending
```

```
System.out.println(Arrays.toString(nums)); // [1, 2, 3, 5, 8, 9]
```

```
Arrays.fill(arr2, 0); // fill all with 0
```

```
int[] copy = Arrays.copyOf(nums, 4); // copy first 4 elements
```

```
int[] rangeCopy = Arrays.copyOfRange(nums, 1, 4); // copy index 1 to 3
```

```
System.out.println(Arrays.binarySearch(nums, 5)); // index of 5 (array must be sorted!)
```

```
int[] a = {1, 2, 3};
```

```
int[] b = {1, 2, 3};
```

```
System.out.println(Arrays.equals(a, b)); // true – element-wise equality
```

```
System.out.println(a == b);           // false – reference comparison
```

```
// 2D array print
```

```
System.out.println(Arrays.deepToString(grid)); // [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
// === COMMON PATTERNS (DSA) ===
```

```
// Initialize array of given size with a value
```

```
int[] dp = new int[10];
```

```
Arrays.fill(dp, Integer.MAX_VALUE); // fill with max int
```

```
// Count frequency
```

```
int[] freq = new int[26]; // for lowercase letters
```

```
String word = "hello";
```

```
for (char ch : word.toCharArray()) {
```

```
    freq[ch - 'a']++;
```

```
}
```

```
// Prefix sum
```

```
int[] original = {1, 2, 3, 4, 5};
```

```
int[] prefix = new int[original.length + 1];
```

```
for (int i = 0; i < original.length; i++) {
```

```
    prefix[i + 1] = prefix[i] + original[i];
```

```
}
```

```
// prefix = [0, 1, 3, 6, 10, 15]
```

```
// sum from index l to r = prefix[r+1] - prefix[l]
```

```
}
```

```
}
```

```
...
```

Key Points

- `arr.length` → FIELD (no parentheses) – for arrays
- `list.size()` → METHOD (with parentheses) – for ArrayList and other Collections
- Arrays are **objects** in Java – they're reference types
- `Arrays.toString(arr)` to print array contents (plain `System.out.println(arr)` prints the reference)
- Java arrays are type-safe – `int[]` can only hold `int`

Common Mistakes

```
```java
```

// Mistake 1 — printing array directly (shows reference, not contents)

```
int[] arr = {1, 2, 3};
```

```
System.out.println(arr); // ❌ [I@6d06d69c (hash code)
```

```
System.out.println(Arrays.toString(arr)); // ✅ [1, 2, 3]
```

// Mistake 2 — using .length() instead of .length for arrays

```
arr.length() // ❌ compile error — .length is a field, not a method
```

```
arr.length // ✅ field access — no parentheses
```

// Mistake 3 — ArrayIndexOutOfBoundsException

```
int[] a = new int[5];
```

```
a[5] = 10; // ❌ valid indices are 0 to 4
```

// Mistake 4 — shallow copy of array

```
int[] original = {1, 2, 3};
```

```
int[] copy = original; // ❌ both point to same array
```

```
copy[0] = 99;
```

```
System.out.println(original[0]); // 99 — original is modified!
```

```
int[] trueCopy = Arrays.copyOf(original, original.length); // ✅ deep copy
```

```
```
```

```
---
```

10. Strings

Explanation

`String` in Java is a **class** (not a primitive), and strings are **immutable** — once created, they cannot be modified. All string methods return new strings. For mutable string operations, use `StringBuilder`.

Examples

```
```java
```

```
public class StringsDemo {
```

```
 public static void main(String[] args) {
```

```
 String s = "Hello, World!";
```

```
 // === LENGTH & ACCESS ===
```

```
 System.out.println(s.length()); // 13 — method, not field!
```

```

System.out.println(s.charAt(0)); // 'H'
System.out.println(s.charAt(s.length() - 1)); // '!'
System.out.println(s.indexOf("World")); // 7 — first occurrence
System.out.println(s.lastIndexOf('l')); // 10
System.out.println(s.contains("World")); // true

// === SUBSTRING ===
System.out.println(s.substring(7)); // "World!" — from index 7 to end
System.out.println(s.substring(7, 12)); // "World" — index 7 to 11 (12 exclusive)

// === CASE ===
System.out.println(s.toUpperCase()); // "HELLO, WORLD!"
System.out.println(s.toLowerCase()); // "hello, world!"

// === TRIM & STRIP ===
String padded = " hello ";
System.out.println(padded.trim()); // "hello" — removes ASCII whitespace
System.out.println(padded.strip()); // "hello" — removes Unicode whitespace (Java 11+, prefer this)

// === REPLACE ===
System.out.println(s.replace('l', 'r')); // "Herro, Worrd!" — replaces all
System.out.println(s.replace("World", "Java")); // "Hello, Java!"
System.out.println(s.replaceAll("[aeiou]", "*")); // regex replace

// === SPLIT ===
String csv = "a,b,c,d";
String[] parts = csv.split(",");
System.out.println(parts[0]); // "a"
System.out.println(parts.length); // 4

String sentence = "Hello World Java";
String[] words = sentence.split(" "); // split by space
String[] limited = sentence.split(" ", 2); // ["Hello", "World Java"]

// === STARTS WITH / ENDS WITH ===
System.out.println(s.startsWith("Hello")); // true
System.out.println(s.endsWith("!")); // true

// === EQUALS ===
String s1 = "hello", s2 = "HELLO";
System.out.println(s1.equals(s2)); // false
System.out.println(s1.equalsIgnoreCase(s2)); // true
System.out.println(s1.compareTo(s2)); // positive — 'h' > 'H' in Unicode

```

```
// === CHAR ARRAY CONVERSION (key for DSA) ===
char[] chars = s.toCharArray(); // String → char array
String back = new String(chars); // char array → String
String fromChars = String.valueOf(chars); // another way

// === String.join / concat ===
String joined = String.join("-", "a", "b", "c"); // "a-b-c"
String[] arr = {"Java", "is", "great"};
String joined2 = String.join(" ", arr); // "Java is great"

// === String.format ===
String formatted = String.format("Name: %s, Age: %d, Score: %.2f", "Alice", 25, 98.6);
System.out.println(formatted); // Name: Alice, Age: 25, Score: 98.60

// === char operations (DSA essentials) ===
char c = 'a';
System.out.println(Character.isLetter(c)); // true
System.out.println(Character.isDigit(c)); // false
System.out.println(Character.isLetterOrDigit(c)); // true
System.out.println(Character.isUpperCase(c)); // false
System.out.println(Character.toLowerCase('A')); // 'a'
System.out.println(Character.toUpperCase('a')); // 'A'
}
}
...
```

### ### Key Points

- `String.length()` → **method** (parentheses) — for String
- `array.length` → **field** (no parentheses) — for arrays
- **Strings are immutable** — `s.replace(...)` returns a NEW string, `s` is unchanged
- For building strings in loops, use `StringBuilder` (much faster — see section 34)
- `String.valueOf(x)` converts any type to String — works for `int`, `char[]`, `Object`, etc.

### ### Common Mistakes

```
```java
// Mistake 1 — comparing strings with ==
String a = new String("hello");
String b = new String("hello");
a == b; // false — different objects
a.equals(b); // true 
```

// Mistake 2 – mutating string (doesn't work!)

```
String s = "hello";
```

```
s.toUpperCase(); // CREATES new string but doesn't modify s
```

```
System.out.println(s); // "hello" – unchanged!
```

```
s = s.toUpperCase(); // ✅ – must reassign
```

// Mistake 3 – String concatenation in loops (very slow!)

```
String result = "";
```

```
for (int i = 0; i < 1000; i++) {
```

```
    result += i; // ❌ creates 1000 new String objects – O(n²)!
```

```
}
```

// Fix: use StringBuilder

```
StringBuilder sb = new StringBuilder();
```

```
for (int i = 0; i < 1000; i++) {
```

```
    sb.append(i); // ✅ O(n)
```

```
}
```

```
String result = sb.toString();
```

// Mistake 4 – NullPointerException

```
String s = null;
```

```
s.length(); // ❌ NullPointerException – check for null first
```

```
if (s != null && s.length() > 0) { } // ✅
```

```
...
```

```
---
```

11. Methods (Functions)

Explanation

In Java, what JavaScript calls "functions" are called **methods**. Every method must be inside a class. Methods must declare their **return type** and **parameter types** explicitly.

Examples

```
```java
```

```
public class Methods {
```

```
 // === Basic method ===
```

```
 // syntax: accessModifier returnType methodName(paramType param, ...) { body }
```

```
 public static int add(int a, int b) {
```

```
 return a + b;
```

```
}
```

```
// === void — no return value ===
```

```
public static void greet(String name) {
 System.out.println("Hello, " + name + "!");
}
```

```
// === Default parameter? Java doesn't have them — use overloading instead ===
```

```
public static void printLine(String message, int times) {
 for (int i = 0; i < times; i++) {
 System.out.println(message);
 }
}
```

```
// Overload with fewer params to simulate default:
```

```
public static void printLine(String message) {
 printLine(message, 1); // calls the other version with default times=1
}
```

```
// === Varargs (variable arguments — like JavaScript rest params) ===
```

```
public static int sum(int... numbers) { // int... is an array internally
 int total = 0;
 for (int n : numbers) total += n;
 return total;
}
```

```
// === Returning arrays ===
```

```
public static int[] twoSum(int[] nums, int target) {
 java.util.Map<Integer, Integer> map = new java.util.HashMap<>();
 for (int i = 0; i < nums.length; i++) {
 int complement = target - nums[i];
 if (map.containsKey(complement)) {
 return new int[]{map.get(complement), i};
 }
 map.put(nums[i], i);
 }
 return new int[]{}; // empty array if no solution
}
```

```
// === Pass by value vs pass by reference ===
```

```
public static void trySwitchPrimitives(int x, int y) {
 int temp = x;
 x = y;
 y = temp;
}
```

```

// modifies LOCAL copies only – original variables unchanged
}

public static void modifyArray(int[] arr) {
 arr[0] = 999; // MODIFIES the original array – array is passed by reference
}

// === Recursion ===
public static int factorial(int n) {
 if (n <= 1) return 1; // base case
 return n * factorial(n - 1); // recursive case
}

public static int fibonacci(int n) {
 if (n <= 1) return n;
 return fibonacci(n - 1) + fibonacci(n - 2);
}

// === Method overloading (same name, different parameters) ===
public static int multiply(int a, int b) { return a * b; }
public static double multiply(double a, double b) { return a * b; }
public static int multiply(int a, int b, int c) { return a * b * c; }

public static void main(String[] args) {
 System.out.println(add(3, 4)); // 7
 greet("Alice"); // Hello, Alice!
 System.out.println(sum(1, 2, 3, 4, 5)); // 15
 System.out.println(factorial(5)); // 120

 // Varargs can accept array too:
 int[] nums = {1, 2, 3};
 System.out.println(sum(nums)); // 6

 // Pass by reference with array:
 int[] arr = {1, 2, 3};
 modifyArray(arr);
 System.out.println(arr[0]); // 999 – original modified!

 // Overloading:
 System.out.println(multiply(3, 4)); // 12 (int version)
 System.out.println(multiply(3.0, 4.0)); // 12.0 (double version)
}
}

```

...

### ### Key Points

- Every method must declare a **return type** (`int`, `String`, `void`, `int[]`, etc.)
- `void` means the method returns nothing
- **Java is pass-by-value** — but for objects/arrays, the "value" is the reference, so object contents CAN be modified inside a method
- **Method overloading**: same method name, different parameter types or count — Java picks the right one at compile time
- **Recursion**: always have a base case to prevent `StackOverflowError`

### ### Common Mistakes

```java

// Mistake 1 — forgetting return statement

```
public static int add(int a, int b) {  
    int sum = a + b;  
    // ❌ forgot return — compile error  
}
```

// Mistake 2 — returning wrong type

```
public static int getDouble(int n) {  
    return n * 2.0; // ❌ compile error — 2.0 is double, method returns int  
    return (int)(n * 2.0); // ✅  
}
```

// Mistake 3 — thinking primitives are modified by methods

```
int x = 5;  
trySwitchPrimitives(x, 10); // x is still 5 after this call
```

// Mistake 4 — `StackOverflowError` (missing base case)

```
public static int infinite(int n) {  
    return infinite(n - 1); // ❌ no base case!  
}  
...
```

12. Object-Oriented Programming — Classes & Objects

Explanation

In Java, **everything lives inside a class**. A class is a blueprint. An **object** is an instance created from that blueprint. This is the heart of Java.

Four pillars of OOP:

1. **Encapsulation** — hiding internal state (private fields, getters/setters)
2. **Inheritance** — child class inherits from parent class (`extends`)
3. **Polymorphism** — same method name, different behavior depending on the object type
4. **Abstraction** — hiding implementation details (abstract classes, interfaces)

Examples

```
```java
// === CLASS DEFINITION ===

public class Car {

 // === FIELDS (instance variables — belong to each object) ===
 String brand;
 String model;
 int year;
 double speed; // current speed

 // === CONSTRUCTOR (special method to create objects) ===
 public Car(String brand, String model, int year) {
 this.brand = brand; // 'this' distinguishes field from parameter
 this.model = model;
 this.year = year;
 this.speed = 0; // default value
 }

 // === METHODS (behaviors of the object) ===
 public void accelerate(double amount) {
 speed += amount;
 System.out.println(brand + " accelerates to " + speed + " km/h");
 }

 public void brake(double amount) {
 speed = Math.max(0, speed - amount); // can't go below 0
 System.out.println(brand + " slows to " + speed + " km/h");
 }

 public String getInfo() {
 return year + " " + brand + " " + model;
 }
}
```

```
// toString() — called automatically when you print an object
@Override
public String toString() {
 return "Car{" + brand + " " + model + " (" + year + "), speed=" + speed + "}";
}
}
```

// === USING THE CLASS ===

```
public class Main {
 public static void main(String[] args) {

 // Create objects (instances) with 'new'
 Car car1 = new Car("Toyota", "Camry", 2022);
 Car car2 = new Car("Tesla", "Model 3", 2023);

 // Access fields
 System.out.println(car1.brand); // Toyota
 System.out.println(car2.year); // 2023

 // Call methods
 car1.accelerate(50); // Toyota accelerates to 50.0 km/h
 car1.accelerate(30); // Toyota accelerates to 80.0 km/h
 car1.brake(20); // Toyota slows to 60.0 km/h

 // toString called automatically:
 System.out.println(car1); // Car{Toyota Camry (2022), speed=60.0}

 // Both objects are independent
 System.out.println(car2.speed); // 0.0 — car2 unaffected

 // null — reference pointing to no object
 Car car3 = null;
 // car3.accelerate(10); // ❌ NullPointerException!
 }
}
...

```

### ### Key Points

- Use `new ClassName(args)` to create an object
- `this` refers to the current object — used to distinguish fields from parameters with same name
- Each object has its own copy of instance fields

- `@Override` annotation marks that you're overriding a method from a parent class (good practice – compiler checks)
- `toString()` is called automatically by `System.out.println(obj)` and `"" + obj`

### ### Common Mistakes

```
```java
// Mistake 1 – forgetting 'new' keyword
Car car = Car("Toyota", "Camry", 2022); // ❌ compile error
Car car = new Car("Toyota", "Camry", 2022); // ✅

// Mistake 2 – NullPointerException
Car car = null;
car.accelerate(50); // ❌ NullPointerException

// Mistake 3 – modifying shared state via reference
Car car1 = new Car("Toyota", "Camry", 2022);
Car car2 = car1; // NOT a copy – both point to same object!
car2.brand = "Honda";
System.out.println(car1.brand); // "Honda" – car1 was affected!
```
```

## ## 13. Constructors

### ### Explanation

A **constructor** is a special method that runs when you create an object with `new`. It initializes the object's state. Java provides a **default constructor** (no args) if you don't write one.

### ### Examples

```
```java
public class Person {
    String name;
    int age;
    String email;

    // === No-arg constructor ===
    public Person() {
        this.name = "Unknown";
        this.age = 0;
    }
}
```

```

        this.email = "";
    }

    // === Parameterized constructor ===
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
        this.email = "";
    }

    // === Full constructor ===
    public Person(String name, int age, String email) {
        this.name = name;
        this.age = age;
        this.email = email;
    }

    // === Constructor chaining with this() ===
    // Call another constructor from this one – must be FIRST statement
    public Person(String name) {
        this(name, 0, ""); // calls the full constructor above
    }

    @Override
    public String toString() {
        return "Person{name=" + name + ", age=" + age + "}";
    }

    public static void main(String[] args) {
        Person p1 = new Person();           // no-arg
        Person p2 = new Person("Alice", 30); // 2-arg
        Person p3 = new Person("Bob", 25, "b@b.com"); // full
        Person p4 = new Person("Charlie");   // chained

        System.out.println(p1); // Person{name=Unknown, age=0}
        System.out.println(p2); // Person{name=Alice, age=30}
    }
}
...

```

Key Points

- Constructors have ****no return type**** (not even ``void``)

- Constructor name must exactly match the class name
- If you define ANY constructor, Java no longer provides the default no-arg constructor — you must define it yourself if needed
- `this(...)` calls another constructor and must be the **first** statement in the constructor body
- `super(...)` calls the parent class constructor — also must be first

14. Encapsulation — Access Modifiers & Getters/Setters

Explanation

Encapsulation means hiding the internal state of an object and controlling access through methods. Use `private` fields and `public` getters/setters.

Access Modifiers:

Modifier	Class	Package	Subclass	World
	-----	-----	-----	-----
<code>public</code>	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✗
(default — no modifier)	✓	✓	✗	✗
<code>private</code>	✓	✗	✗	✗

Examples

```

java
public class BankAccount {

    // === PRIVATE FIELDS — hidden from outside ===
    private String owner;
    private double balance;
    private String accountNumber;

    public BankAccount(String owner, double initialBalance) {
        this.owner = owner;
        this.balance = initialBalance;
        this.accountNumber = generateAccountNumber();
    }

    // === GETTERS — read-only access ===
    public String getOwner() { return owner; }
    public double getBalance() { return balance; }

```

```

public String getAccountNumber() { return accountNumber; }

// === SETTERS — controlled write access with validation ===
public void setOwner(String owner) {
    if (owner == null || owner.isEmpty()) {
        throw new IllegalArgumentException("Owner name cannot be empty");
    }
    this.owner = owner;
}

// === METHODS with business logic ===
public void deposit(double amount) {
    if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
    balance += amount;
}

public void withdraw(double amount) {
    if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
    if (amount > balance) throw new IllegalStateException("Insufficient funds");
    balance -= amount;
}

// === PRIVATE helper — not accessible outside ===
private String generateAccountNumber() {
    return "ACC-" + (int)(Math.random() * 100000);
}

@Override
public String toString() {
    return "BankAccount{owner=" + owner + ", balance=" + balance + "}";
}

}

class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("Alice", 1000.0);

        // account.balance = 99999; // ❌ compile error — private
        System.out.println(account.getBalance()); // ✅ 1000.0

        account.deposit(500);
        System.out.println(account.getBalance()); // 1500.0
    }
}

```

```

account.withdraw(200);
System.out.println(account.getBalance()); // 1300.0

// account.withdraw(99999); // ❌ throws IllegalStateException
}
}
...

```

Key Points

- Rule of thumb: **make fields `private`, methods `public`**
- Getters are named `getX()`, setters are `setX(value)` (Java convention)
- For `boolean`, getters are `isX()` (e.g., `isActive()`, `isEmpty()`)
- Encapsulation allows you to **validate data** and **change implementation** without breaking outside code

15. Inheritance

Explanation

Inheritance allows a child class (`subclass`) to inherit fields and methods from a parent class (`superclass`). Use `extends` keyword.

Examples

```

```java
// === PARENT CLASS ===
public class Animal {
 protected String name;
 protected int age;

 public Animal(String name, int age) {
 this.name = name;
 this.age = age;
 }

 public void eat() {
 System.out.println(name + " is eating");
 }

 public void sleep() {

```

```

 System.out.println(name + " is sleeping");
 }

 public String sound() {
 return "...";
 }

 @Override
 public String toString() {
 return name + " (age " + age + ")";
 }
}

// === CHILD CLASS ===
public class Dog extends Animal {
 private String breed;

 // Constructor — must call super() first
 public Dog(String name, int age, String breed) {
 super(name, age); // calls Animal's constructor
 this.breed = breed;
 }

 // === OVERRIDE parent method ===
 @Override
 public String sound() {
 return "Woof!";
 }

 // === ADD new method (specific to Dog) ===
 public void fetch() {
 System.out.println(name + " is fetching!");
 }

 @Override
 public String toString() {
 return super.toString() + " [" + breed + "]";
 }
}

```

```

public class Cat extends Animal {
 public Cat(String name, int age) {
 super(name, age);
 }
}

```

```

 }

 @Override
 public String sound() {
 return "Meow!";
 }
}

public class Main {
 public static void main(String[] args) {
 Dog dog = new Dog("Rex", 3, "Labrador");
 Cat cat = new Cat("Whiskers", 5);

 dog.eat(); // "Rex is eating" – inherited from Animal
 dog.sleep(); // "Rex is sleeping" – inherited
 dog.fetch(); // "Rex is fetching!" – Dog-specific
 dog.sound(); // "Woof!" – overridden

 System.out.println(dog); // Rex (age 3) [Labrador]

 // === instanceof checks ===
 System.out.println(dog instanceof Dog); // true
 System.out.println(dog instanceof Animal); // true – Dog IS-A Animal
 System.out.println(dog instanceof Cat); // false

 // === Upcasting (safe, automatic) ===
 Animal a = dog; // Dog reference stored as Animal – valid
 a.eat(); // works – eat() is in Animal
 // a.fetch(); // ❌ compile error – Animal doesn't know about fetch()

 // === Downcasting (manual, may throw ClassCastException) ===
 if (a instanceof Dog) { // check first!
 Dog d = (Dog) a; // safe to cast
 d.fetch(); // ✅
 }
 }
}
...

```

### ### Key Points

- Java supports only **\*\*single inheritance\*\*** – a class can extend only ONE class
- But a class can implement **\*\*multiple interfaces\*\*** (see section 17)

- `super.method()` calls the parent's version of a method
- `super(args)` calls the parent constructor — must be the first statement in child constructor
- `@Override` annotation is highly recommended — compiler verifies you're actually overriding
- `protected` fields/methods are accessible by the class AND its subclasses

### Common Mistakes

```
```java
```

```
// Mistake 1 — forgetting super() in child constructor
```

```
public Dog(String name, int age, String breed) {
```

```
    // ❌ if Animal has no no-arg constructor, this will fail
```

```
    this.breed = breed;
```

```
}
```

```
// Fix: always call super() explicitly first if parent has parameterized constructor
```

```
// Mistake 2 — ClassCastException without instanceof check
```

```
Animal a = new Cat("Kitty", 2);
```

```
Dog d = (Dog) a; // ❌ ClassCastException at runtime — Cat is not a Dog!
```

```
// Mistake 3 — using private (not protected) in parent, then accessing in child
```

```
class Animal {
```

```
    private String name; // ❌ child can't access this
```

```
}
```

```
class Dog extends Animal {
```

```
    void test() {
```

```
        System.out.println(name); // ❌ name is private in Animal!
```

```
    }
```

```
}
```

```
// Fix: use protected in Animal, or use getter
```

```
```
```

```

```

## 16. Polymorphism

### Explanation

**Polymorphism** = "many forms." The same method call behaves differently depending on the actual object type at runtime. This is achieved through method overriding and is the cornerstone of flexible, extensible design.

### Examples

```
``java
public class Shape {
 public double area() {
 return 0;
 }

 public void describe() {
 System.out.println("This is a shape with area: " + area());
 }
}
```

```
public class Circle extends Shape {
 private double radius;

 public Circle(double radius) { this.radius = radius; }

 @Override
 public double area() {
 return Math.PI * radius * radius;
 }
}
```

```
public class Rectangle extends Shape {
 private double width, height;

 public Rectangle(double width, double height) {
 this.width = width;
 this.height = height;
 }

 @Override
 public double area() {
 return width * height;
 }
}
```

```
public class Triangle extends Shape {
 private double base, height;

 public Triangle(double base, double height) {
 this.base = base;
 this.height = height;
 }
}
```

```

@Override
public double area() {
 return 0.5 * base * height;
}
}

public class Main {
 public static void main(String[] args) {

 // === POLYMORPHISM in action ===
 Shape[] shapes = {
 new Circle(5),
 new Rectangle(4, 6),
 new Triangle(3, 8)
 };

 for (Shape shape : shapes) {
 shape.describe();
 // JVM decides at RUNTIME which area() to call:
 // Circle → Circle.area(), Rectangle → Rectangle.area(), etc.
 }
 // Output:
 // This is a shape with area: 78.53981633974483
 // This is a shape with area: 24.0
 // This is a shape with area: 12.0

 // === Total area using polymorphism ===
 double totalArea = 0;
 for (Shape s : shapes) {
 totalArea += s.area(); // correct method called for each actual type
 }
 System.out.println("Total area: " + totalArea);
 }
}
...

```

### ### Key Points

- **Compile-time polymorphism** = method overloading (resolved by compiler)
- **Runtime polymorphism** = method overriding (resolved at runtime by JVM)
- The actual method that runs depends on the **object's actual type**, not the reference type
- This lets you write code that works with the base type (`Shape``) and handles all subtypes automatically

## ## 17. Abstraction — Abstract Classes & Interfaces

### ### Explanation

**Abstraction** means defining WHAT something should do, without specifying HOW. Two mechanisms:

1. **Abstract class** — a class that cannot be instantiated; may have abstract (unimplemented) methods AND regular methods
2. **Interface** — a pure contract: defines method signatures; classes `implement` it

### ### Examples

```
```java
// === ABSTRACT CLASS ===
public abstract class Vehicle {
    protected String brand;
    protected int year;

    public Vehicle(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }

    // Abstract method — no body — subclasses MUST implement
    public abstract void startEngine();
    public abstract double fuelEfficiency(); // km per liter

    // Concrete method — subclasses inherit this
    public void displayInfo() {
        System.out.println(brand + " (" + year + ") — " + fuelEfficiency() + " km/L");
    }
}

public class ElectricCar extends Vehicle {
    private int batteryCapacity;

    public ElectricCar(String brand, int year, int batteryCapacity) {
        super(brand, year);
        this.batteryCapacity = batteryCapacity;
    }
}
```

```

@Override
public void startEngine() {
    System.out.println(brand + " silently starts (electric)");
}

@Override
public double fuelEfficiency() {
    return batteryCapacity * 6.5; // km per kWh * capacity — simplified
}
}

// === INTERFACE ===
public interface Flyable {
    // All fields in interface are public static final (constants)
    int MAX_ALTITUDE = 50000; // implicitly public static final

    // All methods are public abstract (unless default or static)
    void fly();
    void land();

    // DEFAULT method (Java 8+) — has implementation; classes can override
    default void describe() {
        System.out.println("This object can fly!");
    }

    // STATIC method in interface
    static boolean canFly(Object obj) {
        return obj instanceof Flyable;
    }
}

public interface Swimmable {
    void swim();
}

// A class can implement MULTIPLE interfaces
public class Duck extends Animal implements Flyable, Swimmable {
    public Duck(String name) { super(name, 1); }

    @Override public void fly() { System.out.println(name + " flaps and flies!"); }
    @Override public void land() { System.out.println(name + " lands on water"); }
    @Override public void swim() { System.out.println(name + " paddles around"); }
}

```

```

@Override public String sound() { return "Quack!"; }
}

public class Main {
    public static void main(String[] args) {
        // Vehicle v = new Vehicle("X", 2020); // ❌ can't instantiate abstract class

        ElectricCar tesla = new ElectricCar("Tesla", 2023, 100);
        tesla.startEngine(); // Tesla silently starts (electric)
        tesla.displayInfo(); // Tesla (2023) — 650.0 km/L

        Duck duck = new Duck("Donald");
        duck.fly(); // Donald flaps and flies!
        duck.swim(); // Donald paddles around
        duck.describe();// This object can fly! (default method)

        // Polymorphism with interfaces:
        Flyable f = duck;
        f.fly(); // works!

        // f.swim(); // ❌ Flyable doesn't know about swim()
    }
}
...

```

Abstract Class vs Interface — when to use which?

Abstract Class	Interface
Inheritance `extends` (one only)	`implements` (multiple!)
Constructor ☒ can have	☐ cannot
Fields Any type	Only `public static final`
Methods Abstract + concrete	Abstract + default + static
Use when Shared base class with common state	Defining a contract / capability

Key Points

- Abstract class: **"IS-A"** relationship with shared code
- Interface: **"CAN-DO"** relationship (Flyable, Serializable, Comparable)
- A class can `extends` one class but `implements` many interfaces
- If a class doesn't implement all abstract methods from parent/interface, it must also be `abstract`

18. Static Keyword

Explanation

`static` means the member belongs to the **class itself**, not to any specific instance. Shared across all objects of that class.

Examples

```
```java
public class Counter {

 // === STATIC FIELD — shared across ALL instances ===
 private static int count = 0;

 // === INSTANCE FIELD — unique to each object ===
 private int id;
 private String name;

 public Counter(String name) {
 count++; // increment shared counter
 this.id = count; // each gets a unique ID
 this.name = name;
 }

 // === STATIC METHOD — no 'this', no instance fields ===
 public static int getCount() {
 return count;
 // return name; // ❌ can't access instance field from static method
 }

 // === INSTANCE METHOD — has access to 'this' and instance fields ===
 public String getInfo() {
 return "Counter #" + id + ": " + name + " (total: " + count + ")";
 }

 // === STATIC CONSTANT ===
 public static final double MAX_LOAD = 0.75;

 // === STATIC NESTED CLASS ===
 public static class Helper {
 public static String format(int n) {
```

```

 return "Count: " + n;
 }
}

// === STATIC INITIALIZER BLOCK — runs once when class is loaded ===
static {
 System.out.println("Counter class loaded");
 count = 0;
}

public static void main(String[] args) {
 System.out.println(Counter.getCount()); // 0 — access via class name

 Counter c1 = new Counter("Alice");
 Counter c2 = new Counter("Bob");
 Counter c3 = new Counter("Charlie");

 System.out.println(Counter.getCount()); // 3
 System.out.println(c1.getInfo()); // Counter #1: Alice (total: 3)
 System.out.println(c2.getInfo()); // Counter #2: Bob (total: 3)

 // Access static member via instance — works but bad style!
 System.out.println(c1.getCount()); // works but use Counter.getCount()

 // Static utility class (like Math)
 System.out.println(Math.abs(-5)); // 5
 System.out.println(Math.max(10, 20)); // 20
 System.out.println(Math.sqrt(16)); // 4.0
 System.out.println(Math.floor(3.9)); // 3.0
 System.out.println(Math.ceil(3.1)); // 4.0
 System.out.println(Math.min(5, 3)); // 3
 System.out.println(Math.log(Math.E)); // 1.0
}
}
...

```

### ### Key Points

- ``static`` methods can ONLY access ``static`` fields and methods — not instance members
- Call static methods via class name: ``Counter.getCount()``, not ``obj.getCount()``
- ``static final`` = constant: by convention, use ALL\_CAPS\_SNAKE\_CASE
- ``Math`` class is all static methods — you never create ``new Math()``
- ``main`` is static so JVM can call it without creating an object

## ## 19. final Keyword

### ### Explanation

`final` has three uses in Java:

1. **`final` variable** → cannot be reassigned (like `const` in JavaScript)
2. **`final` method** → cannot be overridden by subclasses
3. **`final` class** → cannot be extended (subclassed)

### ### Examples

```
```java
```

```
// final class — cannot be extended (String is final, for example)
```

```
public final class ImmutablePoint {
```

```
    // final fields — must be initialized once (in declaration or constructor)
```

```
    private final int x;
```

```
    private final int y;
```

```
    public ImmutablePoint(int x, int y) {
```

```
        this.x = x;
```

```
        this.y = y;
```

```
        // After constructor, x and y can never change
```

```
    }
```

```
    public int getX() { return x; }
```

```
    public int getY() { return y; }
```

```
    // final method — subclasses can't override (but class is final anyway here)
```

```
    public final double distanceTo(ImmutablePoint other) {
```

```
        int dx = this.x - other.x;
```

```
        int dy = this.y - other.y;
```

```
        return Math.sqrt(dx * dx + dy * dy);
```

```
    }
```

```
}
```

```
public class FinalDemo {
```

```
    public static void main(String[] args) {
```

```
        // final local variable
```

```
        final int MAX = 100;
```

// MAX = 200; // ❌ compile error — cannot reassign final variable

// final reference — reference is fixed, but object content CAN change

```
final int[] arr = {1, 2, 3};
```

```
arr[0] = 99; // ✅ modifying content is fine
```

```
// arr = new int[]{4, 5, 6}; // ❌ cannot reassign the reference
```

```
ImmutablePoint p1 = new ImmutablePoint(0, 0);
```

```
ImmutablePoint p2 = new ImmutablePoint(3, 4);
```

```
System.out.println(p1.distanceTo(p2)); // 5.0
```

```
}
```

```
}
```

```
...
```

Key Points

- `String` in Java is `final` — that's why you can't subclass it
- `final` variable ≠ immutable content; it means the **reference** can't point to something else
- Use `final` on fields to create immutable-ish objects (good for thread safety, constants)

20. Exception Handling

Explanation

Exceptions are runtime errors. Java forces you to handle them (for **checked exceptions**) or lets you optionally handle them (for **unchecked exceptions**).

Exception hierarchy:

...

Throwable

├── Error (serious system errors — don't catch: OutOfMemoryError, StackOverflowError)

└── Exception

 ├── RuntimeException (unchecked — optional to handle)

 ├── NullPointerException

 ├── ArrayIndexOutOfBoundsException

 ├── ClassCastException

 ├── NumberFormatException

 ├── IllegalArgumentException

 └── ArithmeticException (divide by zero)

 └── IOException (checked — MUST handle or declare)

 └── SQLException (checked)

Examples

```
```java
```

```
import java.util.InputMismatchException;
import java.util.Scanner;
```

```
public class ExceptionHandling {
```

```
 public static void main(String[] args) {
```

```
 // === TRY-CATCH-FINALLY ===
```

```
 try {
```

```
 int result = 10 / 0; // throws ArithmeticException
```

```
 System.out.println(result); // never reached
```

```
 } catch (ArithmeticException e) {
```

```
 System.out.println("Error: " + e.getMessage()); // / by zero
```

```
 } finally {
```

```
 System.out.println("This always runs"); // cleanup code here
```

```
 }
```

```
 // === MULTIPLE CATCH BLOCKS ===
```

```
 try {
```

```
 String s = null;
```

```
 s.length(); // NullPointerException
```

```
 } catch (NullPointerException e) {
```

```
 System.out.println("Null pointer: " + e.getMessage());
```

```
 } catch (Exception e) {
```

```
 System.out.println("General exception: " + e.getMessage());
```

```
 }
```

```
 // Order matters: more specific exceptions MUST come before general ones
```

```
 // === MULTI-CATCH (Java 7+) ===
```

```
 try {
```

```
 String[] arr = new String[3];
```

```
 int x = Integer.parseInt(arr[5]); // ArrayIndexOutOfBoundsException
```

```
 } catch (ArrayIndexOutOfBoundsException | NumberFormatException e) {
```

```
 System.out.println("Caught: " + e.getClass().getSimpleName());
```

```
 }
```

```
 // === THROW — explicitly throw an exception ===
```

```
 try {
```

```

 validateAge(-5);
 } catch (IllegalArgumentException e) {
 System.out.println("Validation error: " + e.getMessage());
 }

 // === TRY-WITH-RESOURCES (auto-close resources — Java 7+) ===
 // try (Scanner sc = new Scanner(System.in)) {
 // String line = sc.nextLine();
 // } // sc automatically closed — no need for finally block
}

// === THROWING EXCEPTIONS ===
public static void validateAge(int age) {
 if (age < 0) {
 throw new IllegalArgumentException("Age cannot be negative: " + age);
 }
 if (age > 150) {
 throw new IllegalArgumentException("Age is unrealistically high: " + age);
 }
}

// === CHECKED EXCEPTION — must declare with 'throws' ===
public static String readFile(String path) throws java.io.IOException {
 // If you don't handle IOException here, you must declare 'throws IOException'
 java.io.BufferedReader reader = new java.io.BufferedReader(
 new java.io.FileReader(path)
);
 return reader.readLine();
}

// === CUSTOM EXCEPTION ===
public static class InsufficientFundsException extends RuntimeException {
 private double amount;

 public InsufficientFundsException(double amount) {
 super("Insufficient funds. Needed: " + amount);
 this.amount = amount;
 }

 public double getAmount() { return amount; }
}
}
...

```

### ### Key Points

- **Checked exceptions**: must be caught OR declared with `throws` — `IOException`, `SQLException`
- **Unchecked (Runtime) exceptions**: optional to catch — `NullPointerException`, `ArithmeticException`
- `finally` block always runs — use for cleanup (closing connections, etc.)
- `try-with-resources` is the modern way to handle closeable resources — auto-calls `.close()`
- Catch specific exceptions first, general ones (`Exception`) last

### ### Common Mistakes

```
```java
// Mistake 1 — catching general Exception when you should be specific
try {
    riskyCode();
} catch (Exception e) { // ❌ catches everything — hides bugs
    // what actually failed?
}
```

```
// Mistake 2 — swallowing exceptions (empty catch block)
try {
    riskyCode();
} catch (Exception e) {
    // ❌ silently ignoring the exception — terrible practice!
}

// Always at minimum: e.printStackTrace() or log the error
```

```
// Mistake 3 — returning in finally (overrides return in try)
try {
    return 1;
} finally {
    return 2; // ❌ returns 2, ignoring the try's return 1
}
...
---
```

21. Generics

Explanation

Generics let you write type-safe code that works with any type. Instead of writing the same logic for `int`, `String`, `Double` etc., you write it once with a type parameter `<T>`.

This is how `ArrayList<String>`, `HashMap<String, Integer>` etc. work.

Examples

```
```java
```

```
// === GENERIC CLASS ===
```

```
public class Pair<A, B> {
 private A first;
 private B second;

 public Pair(A first, B second) {
 this.first = first;
 this.second = second;
 }

 public A getFirst() { return first; }
 public B getSecond() { return second; }
```

```
@Override
```

```
 public String toString() {
 return "(" + first + ", " + second + ")";
 }
}
```

```
// === GENERIC METHOD ===
```

```
public class GenericUtils {
```

```
 // <T> before return type declares the type parameter
```

```
 public static <T> void swap(T[] arr, int i, int j) {
 T temp = arr[i];
 arr[i] = arr[j];
 arr[j] = temp;
 }
```

```
 public static <T extends Comparable<T>> T max(T a, T b) {
 return a.compareTo(b) >= 0 ? a : b; // works for any Comparable type
 }
```

```
 public static <T> void printArray(T[] arr) {
 for (T item : arr) {
 System.out.print(item + " ");
 }
```

```

 System.out.println();
 }
}

public class Main {
 public static void main(String[] args) {
 // Generic class usage
 Pair<String, Integer> pair = new Pair<>("Alice", 30);
 System.out.println(pair.getFirst()); // Alice
 System.out.println(pair.getSecond()); // 30
 System.out.println(pair); // (Alice, 30)

 Pair<Double, Boolean> pair2 = new Pair<>(3.14, true);

 // Generic methods
 Integer[] nums = {3, 1, 4, 1, 5};
 GenericUtils.swap(nums, 0, 4);
 GenericUtils.printArray(nums); // 5 1 4 1 3

 System.out.println(GenericUtils.max(10, 20)); // 20
 System.out.println(GenericUtils.max("apple", "banana")); // banana

 // Why generics? Type safety at compile time:
 // Without generics:
 java.util.List rawList = new java.util.ArrayList();
 rawList.add("hello");
 rawList.add(42); // ❌ can mix types — runtime error possible

 // With generics:
 java.util.List<String> typedList = new java.util.ArrayList<>();
 typedList.add("hello");
 // typedList.add(42); // ❌ compile error — caught early!
 }
}
...

```

### ### Key Points

- ``<T>`` is a type parameter — any letter works, but conventions: ``T`` (Type), ``E`` (Element), ``K`` (Key), ``V`` (Value), ``N`` (Number)
- ``<T extends Comparable<T>>`` = bounded type parameter (T must implement Comparable)
- Diamond operator ``<>`` in ``new ArrayList<>()`` lets the compiler infer the type
- Generics are **erased at runtime** (type erasure) — they're a compile-time safety feature

## ## 22. Collections Framework

### ### Explanation

Java's Collections Framework provides ready-made, generic data structures. This is your toolbox for DSA.

**Overview:**

Collection (interface)

- ├── List (interface) – ordered, allows duplicates
  - │ ├── ArrayList – dynamic array (fast random access)
  - │ ├── LinkedList – doubly linked list (fast insert/delete at ends)
  - │ └── Stack – LIFO (legacy – prefer Deque)
- ├── Set (interface) – no duplicates
  - │ ├── HashSet – unordered,  $O(1)$  operations
  - │ ├── LinkedHashSet – insertion-ordered,  $O(1)$  operations
  - │ └── TreeSet – sorted order,  $O(\log n)$  operations
- └── Queue (interface) – FIFO
  - │ ├── LinkedList – also implements Queue
  - │ ├── PriorityQueue – min-heap by default
  - └── ArrayDeque – double-ended queue (fast, preferred)

Map (separate interface – key-value pairs)

- ├── HashMap – unordered,  $O(1)$  average
- ├── LinkedHashMap – insertion-ordered
- └── TreeMap – sorted by keys,  $O(\log n)$

### ### Key Points

- Use `List<Integer>` not `ArrayList<Integer>` for variable types when possible – code to the interface
- For DSA: `HashMap` + `HashSet` for  $O(1)$  lookup; `PriorityQueue` for heap; `Deque` for stack/queue
- **All Collections use wrapper types** (object types) – `Integer`, not `int`; `Character`, not `char`
- Autoboxing converts automatically: `list.add(5)` → Java converts `5` (int) to `Integer(5)` automatically

## ## 23. ArrayList

### ### Explanation

`ArrayList` is a **dynamic array** — it automatically resizes when needed. It's the most-used Collection in Java, equivalent to JavaScript arrays for most purposes.

### ### Examples

```
```java
import java.util.*;

public class ArrayListDemo {
    public static void main(String[] args) {

        // === CREATION ===
        ArrayList<Integer> list = new ArrayList<>();    // empty list
        ArrayList<String> names = new ArrayList<>(10); // initial capacity hint
        ArrayList<Integer> fromArr = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
        List<String> immutable = List.of("a", "b", "c"); // immutable list (Java 9+)

        // === ADDING ===
        list.add(10);    // add to end
        list.add(20);
        list.add(30);
        list.add(1, 15); // insert at index 1 → [10, 15, 20, 30]
        list.addAll(Arrays.asList(40, 50)); // add collection

        // === ACCESSING ===
        System.out.println(list.get(0)); // 10
        System.out.println(list.get(list.size() - 1)); // 50 — last element
        System.out.println(list.size()); // 6
        System.out.println(list.isEmpty()); // false

        // === MODIFYING ===
        list.set(0, 99); // replace index 0 with 99

        // === REMOVING ===
        list.remove(0);    // remove by INDEX — removes 99 → [15, 20, 30, 40, 50]
        list.remove(Integer.valueOf(20)); // remove by VALUE — removes the element 20
        // ⚠ list.remove(20) — if list is List<Integer>, 20 is autoboxed — removes value 20, not index 20
        // BUT if ambiguous, use Integer.valueOf(20) to be explicit about value removal

        // === SEARCHING ===
        System.out.println(list.contains(30)); // true
    }
}
```

```
System.out.println(list.indexOf(30)); // index of 30
```

```
// === SORTING ===
```

```
List<Integer> nums = new ArrayList<>(Arrays.asList(5, 3, 8, 1, 9, 2));
```

```
Collections.sort(nums);           // ascending: [1, 2, 3, 5, 8, 9]
```

```
Collections.sort(nums, Collections.reverseOrder()); // descending
```

```
nums.sort((a, b) -> a - b);       // ascending with lambda
```

```
nums.sort((a, b) -> b - a);       // descending with lambda
```

```
System.out.println(nums);
```

```
// === ITERATION ===
```

```
for (int n : nums) System.out.print(n + " ");
```

```
nums.forEach(n -> System.out.print(n + " ")); // lambda forEach
```

```
for (int i = 0; i < nums.size(); i++) {  
    System.out.println(i + ": " + nums.get(i));  
}
```

```
// === CONVERTING ===
```

```
// List → array
```

```
Integer[] arr = nums.toArray(new Integer[0]);
```

```
// Array → List
```

```
List<Integer> fromList = Arrays.asList(arr); // fixed size!
```

```
List<Integer> modifiable = new ArrayList<>(Arrays.asList(arr)); // mutable
```

```
// === SUBLIST ===
```

```
List<Integer> sub = nums.subList(1, 4); // index 1 to 3 (4 exclusive)
```

```
// === USEFUL OPERATIONS ===
```

```
Collections.reverse(nums);
```

```
Collections.shuffle(nums);
```

```
System.out.println(Collections.max(nums));
```

```
System.out.println(Collections.min(nums));
```

```
Collections.fill(nums, 0); // fill all with 0
```

```
}
```

```
}
```

```
...
```

Key Points

- `ArrayList.get(i)` → $O(1)$ random access (like array)
- `ArrayList.add(i, val)` → $O(n)$ insert at middle (shifts elements)
- `ArrayList.remove(i)` → $O(n)$ (shifts elements)
- `ArrayList.add(val)` at end → $O(1)$ amortized
- Use `ArrayList` when you need frequent random access; `LinkedList` for frequent insertions/deletions at ends

Common Mistakes

```
```java
```

```
// Mistake 1 — remove(int) vs remove(Object) ambiguity
```

```
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
```

```
list.remove(1); // removes INDEX 1 → [1, 3, 4, 5]
```

```
list.remove(Integer.valueOf(1)); // removes VALUE 1 → [3, 4, 5]
```

```
// Mistake 2 — ConcurrentModificationException (modifying while iterating)
```

```
for (Integer n : list) {
```

```
 if (n == 3) list.remove(n); // ❌ ConcurrentModificationException
```

```
}
```

```
// Fix: use Iterator
```

```
Iterator<Integer> it = list.iterator();
```

```
while (it.hasNext()) {
```

```
 if (it.next() == 3) it.remove(); // ✅
```

```
}
```

```
// Or: list.removeIf(n -> n == 3); // ✅ Java 8+
```

```
// Mistake 3 — Arrays.asList returns fixed-size list
```

```
List<Integer> fixed = Arrays.asList(1, 2, 3);
```

```
fixed.add(4); // ❌ UnsupportedOperationException
```

```
List<Integer> mutable = new ArrayList<>(Arrays.asList(1, 2, 3)); // ✅
```

```
...
```

```

```

## ## 24. LinkedList

### Explanation

`LinkedList` implements both `List` and `Deque`. It's a doubly linked list — each node holds a reference to previous and next. Fast  $O(1)$  at both ends; slow  $O(n)$  random access.

### Examples

```

```java
import java.util.*;

public class LinkedListDemo {
    public static void main(String[] args) {

        LinkedList<Integer> ll = new LinkedList<>();

        // === AS A LIST ===
        ll.add(1); ll.add(2); ll.add(3);
        System.out.println(ll); // [1, 2, 3]

        // === AS A DEQUE (double-ended queue) ===
        ll.addFirst(0); // add to front → [0, 1, 2, 3]
        ll.addLast(4); // add to back → [0, 1, 2, 3, 4]
        ll.offerFirst(-1); // add to front (returns boolean) → [-1, 0, 1, 2, 3, 4]
        ll.offerLast(5); // add to back

        System.out.println(ll.getFirst()); // -1 — peek at front
        System.out.println(ll.getLast()); // 5 — peek at back
        System.out.println(ll.peekFirst()); // -1 — returns null if empty (safe)

        ll.removeFirst(); // remove front element
        ll.removeLast(); // remove back element
        ll.pollFirst(); // remove front, returns null if empty (safe)
        ll.pollLast(); // remove back, returns null if empty (safe)

        System.out.println(ll); // [0, 1, 2, 3, 4]

        // === WHEN TO USE LinkedList ===
        // ✅ As a Queue: addLast() + removeFirst()
        // ✅ As a Stack: addFirst() + removeFirst()
        // ✅ Frequent insertions/deletions at BOTH ends
        // ❌ Random access (ll.get(i) is O(n) — very slow)
        // Prefer ArrayDeque for Stack/Queue in practice (faster)
    }
}
...

```

Explanation

A **Stack** is LIFO (Last In, First Out). Java has an old `Stack` class, but **prefer `Deque` (specifically `ArrayDeque`)** which is faster.

Examples

```
```java
```

```
import java.util.*;
```

```
public class StackDemo {
 public static void main(String[] args) {

 // === PREFERRED: Use Deque as Stack ===
 Deque<Integer> stack = new ArrayDeque<>();

 stack.push(1); // push to top (addFirst)
 stack.push(2);
 stack.push(3);
 System.out.println(stack); // [3, 2, 1] – top is at front

 System.out.println(stack.peek()); // 3 – peek top (no remove)
 System.out.println(stack.pop()); // 3 – remove and return top
 System.out.println(stack.pop()); // 2
 System.out.println(stack.isEmpty()); // false
 System.out.println(stack.size()); // 1

 // === DSA patterns with stack ===

 // Pattern 1: Valid Parentheses
 System.out.println(isValid("{}")); // true
 System.out.println(isValid("(")); // false

 // Pattern 2: Reverse a string using stack
 String original = "hello";
 Deque<Character> charStack = new ArrayDeque<>();
 for (char c : original.toCharArray()) charStack.push(c);
 StringBuilder reversed = new StringBuilder();
 while (!charStack.isEmpty()) reversed.append(charStack.pop());
 System.out.println(reversed.toString()); // "olleh"
 }

 static boolean isValid(String s) {
```

```

Deque<Character> stack = new ArrayDeque<>();
for (char c : s.toCharArray()) {
 if (c == '(' || c == '[' || c == '{') {
 stack.push(c);
 } else {
 if (stack.isEmpty()) return false;
 char top = stack.pop();
 if (c == ')' && top != '(') return false;
 if (c == ']' && top != '[') return false;
 if (c == '}' && top != '{') return false;
 }
}
return stack.isEmpty();
}
}
...

```

### ### Key Points

- **`ArrayDeque`** as **`Stack`** is preferred over the legacy `Stack` class (faster, no synchronized overhead)
- `push(x)` → adds to top | `pop()` → removes from top | `peek()` → looks at top without removing
- `pop()` and `peek()` throw `NoSuchElementException` if empty — use `pollFirst()` / `peekFirst()` which return `null` if empty

---

## ## 26. Queue & Deque

### ### Explanation

A **`Queue`** is FIFO (First In, First Out). Java's `Queue` interface has multiple implementations. **`ArrayDeque`** is the best general-purpose choice.

### ### Examples

```

```java
import java.util.*;

public class QueueDemo {
    public static void main(String[] args) {

        // === QUEUE (FIFO) ===
        Queue<Integer> queue = new ArrayDeque<>();
    }
}

```

```

queue.offer(1); // add to back (returns false if fails – safe)
queue.offer(2);
queue.offer(3);
// queue.add(4); // also adds to back, throws exception if fails

```

```

System.out.println(queue.peek()); // 1 – front element (no remove, null if empty)
System.out.println(queue.poll()); // 1 – remove and return front (null if empty)
System.out.println(queue.poll()); // 2
System.out.println(queue); // [3]
System.out.println(queue.size()); // 1

```

```

// === DEQUE (Double-Ended Queue) ===
Deque<Integer> deque = new ArrayDeque<>();

```

```

deque.offerFirst(1); // add to front
deque.offerLast(2); // add to back
deque.offerFirst(0); // [0, 1, 2]

```

```

System.out.println(deque.peekFirst()); // 0
System.out.println(deque.peekLast()); // 2
System.out.println(deque.pollFirst()); // 0 – remove from front
System.out.println(deque.pollLast()); // 2 – remove from back
System.out.println(deque); // [1]

```

```

// === DSA: BFS with Queue ===
// See Part 2 for full BFS template

```

```

// === DSA: Sliding Window Maximum with Deque ===
// Monotonic deque stores indices in decreasing order of values
// Front = index of max element in current window

```

```

}
}
...

```

```

---
```

27. HashMap & HashSet

Explanation

****HashMap**** → stores key-value pairs with $O(1)$ average get/put/remove. Keys must be unique.

****HashSet**** → stores unique elements with $O(1)$ average add/contains/remove. Internally a HashMap.

Examples

```
```java
import java.util.*;

public class HashMapDemo {
 public static void main(String[] args) {

 // === HASHMAP ===
 HashMap<String, Integer> map = new HashMap<>();

 // === PUTTING ===
 map.put("Alice", 90);
 map.put("Bob", 85);
 map.put("Charlie", 92);
 map.put("Alice", 95); // overwrites existing key

 // === GETTING ===
 System.out.println(map.get("Alice")); // 95
 System.out.println(map.get("Unknown")); // null – key doesn't exist
 System.out.println(map.getDefault("Unknown", 0)); // 0 – safe default

 // === CHECKING ===
 System.out.println(map.containsKey("Bob")); // true
 System.out.println(map.containsValue(92)); // true
 System.out.println(map.size()); // 3
 System.out.println(map.isEmpty()); // false

 // === REMOVING ===
 map.remove("Bob");
 System.out.println(map); // {Alice=95, Charlie=92}

 // === ITERATING ===
 // Keys only:
 for (String key : map.keySet()) {
 System.out.println(key);
 }
 // Values only:
 for (int val : map.values()) {
 System.out.println(val);
 }
 // Key-value pairs (most common):
```

```

for (Map.Entry<String, Integer> entry : map.entrySet()) {
 System.out.println(entry.getKey() + " → " + entry.getValue());
}
// Lambda forEach:
map.forEach((key, val) -> System.out.println(key + ": " + val));

```

// === COMMON DSA PATTERNS ===

```

// Frequency counter
String text = "hello world";
Map<Character, Integer> freq = new HashMap<>();
for (char c : text.toCharArray()) {
 freq.put(c, freq.getOrDefault(c, 0) + 1);
}
System.out.println(freq); // {h=1, e=1, l=3, o=2, =1, w=1, r=1, d=1}

```

```

// Increment shorthand:
freq.merge('a', 1, Integer::sum); // add 1 to existing or put 1 if absent

```

```

// Put if absent:
freq.putIfAbsent('z', 0);

```

```

// Compute:
freq.compute('h', (k, v) -> v == null ? 1 : v + 1); // h becomes 2

```

```

// Group elements:
String[] words = {"eat", "tea", "tan", "ate", "nat", "bat"};
Map<String, List<String>> groups = new HashMap<>();
for (String word : words) {
 char[] chars = word.toCharArray();
 Arrays.sort(chars);
 String key = new String(chars);
 groups.computeIfAbsent(key, k -> new ArrayList<>()).add(word);
}
System.out.println(groups); // anagram groups

```

// =====

```

// === HASHSET ===
HashSet<Integer> set = new HashSet<>();

```

```

set.add(1);
set.add(2);

```

```

set.add(3);
set.add(2); // duplicate — ignored

System.out.println(set); // [1, 2, 3] (order not guaranteed!)
System.out.println(set.contains(2)); // true
System.out.println(set.contains(5)); // false
set.remove(2);
System.out.println(set.size()); // 2

```

// Set operations:

```

Set<Integer> setA = new HashSet<>(Arrays.asList(1, 2, 3, 4));
Set<Integer> setB = new HashSet<>(Arrays.asList(3, 4, 5, 6));

```

```

Set<Integer> intersection = new HashSet<>(setA);
intersection.retainAll(setB); // {3, 4}

```

```

Set<Integer> union = new HashSet<>(setA);
union.addAll(setB); // {1, 2, 3, 4, 5, 6}

```

```

Set<Integer> difference = new HashSet<>(setA);
difference.removeAll(setB); // {1, 2}

```

// Convert array to Set (remove duplicates):

```

int[] arr = {1, 2, 2, 3, 3, 4};
Set<Integer> unique = new HashSet<>();
for (int n : arr) unique.add(n);
// Or: Set<Integer> unique = new HashSet<>(Arrays.asList(1,2,2,3,3,4));

```

```

}

```

```

}

```

```

...

```

### ### Key Points

- `HashMap` allows ONE null key and multiple null values
- `HashMap` is **unordered** — use `LinkedHashMap` for insertion order, `TreeMap` for sorted order
- `HashSet` is backed by `HashMap` (keys only, values are a dummy object)
- For DSA: HashMap is your O(1) lookup tool — used in 50%+ of medium problems

```

```

## ## 28. TreeMap & TreeSet

### ### Explanation

`TreeMap` and `TreeSet` maintain **sorted order** (natural ordering or custom Comparator). Based on Red-Black tree —  $O(\log n)$  operations.

### ### Examples

```
```java
import java.util.*;

public class TreeDemo {
    public static void main(String[] args) {

        // === TREEMAP — sorted by keys ===
        TreeMap<String, Integer> tmap = new TreeMap<>();
        tmap.put("banana", 2);
        tmap.put("apple", 5);
        tmap.put("cherry", 1);
        tmap.put("date", 3);

        System.out.println(tmap); // {apple=5, banana=2, cherry=1, date=3} — alphabetical!

        // Navigation methods (unique to TreeMap):
        System.out.println(tmap.firstKey());    // apple
        System.out.println(tmap.lastKey());     // date
        System.out.println(tmap.floorKey("c")); // banana — largest key ≤ "c"
        System.out.println(tmap.ceilingKey("c")); // cherry — smallest key ≥ "c"
        System.out.println(tmap.lowerKey("cherry")); // banana — strictly less than
        System.out.println(tmap.higherKey("cherry")); // date — strictly greater than

        // Range view:
        System.out.println(tmap.subMap("b", "d")); // {banana=2, cherry=1}
        System.out.println(tmap.headMap("cherry")); // {apple=5, banana=2}
        System.out.println(tmap.tailMap("cherry")); // {cherry=1, date=3}

        // === TREESSET — sorted unique elements ===
        TreeSet<Integer> tset = new TreeSet<>();
        tset.add(5); tset.add(3); tset.add(8); tset.add(1); tset.add(9);

        System.out.println(tset);    // [1, 3, 5, 8, 9] — sorted!
        System.out.println(tset.first()); // 1
        System.out.println(tset.last());  // 9
        System.out.println(tset.floor(6)); // 5 — largest ≤ 6
        System.out.println(tset.ceiling(6)); // 8 — smallest ≥ 6
    }
}
```

```
System.out.println(tset.headSet(5)); // [1, 3] – less than 5
```

```
System.out.println(tset.tailSet(5)); // [5, 8, 9] –  $\geq 5$ 
```

```
// Reverse order:
```

```
TreeSet<Integer> reversed = new TreeSet<>(Collections.reverseOrder());
```

```
reversed.addAll(tset);
```

```
System.out.println(reversed); // [9, 8, 5, 3, 1]
```

```
}
```

```
}
```

```
...
```

Key Points

- Use `TreeMap`/`TreeSet` when you need sorted order AND range queries
- $O(\log n)$ for all operations (vs HashMap's $O(1)$ average)
- `floor(x)` $\rightarrow$ largest element $\leq x$; `ceiling(x)` $\rightarrow$ smallest element $\geq x$
- For DSA: useful in "nearest value", "sliding window with sorted elements", "order statistics" problems

```
---
```

29. PriorityQueue (Heap)

Explanation

Java's `PriorityQueue` is a **min-heap** by default — the smallest element is always at the front. Use `Collections.reverseOrder()` or a custom comparator for a max-heap.

Examples

```
```java
```

```
import java.util.*;
```

```
public class PriorityQueueDemo {
 public static void main(String[] args) {
```

```
 // === MIN-HEAP (default) ===
```

```
 PriorityQueue<Integer> minHeap = new PriorityQueue<>();
```

```
 minHeap.offer(5);
```

```
 minHeap.offer(3);
```

```
 minHeap.offer(8);
```

```
 minHeap.offer(1);
```

```
 minHeap.offer(9);
```

```

System.out.println(minHeap.peek()); // 1 – minimum element
while (!minHeap.isEmpty()) {
 System.out.print(minHeap.poll() + " "); // 1 3 5 8 9 – sorted output
}
System.out.println();

// === MAX-HEAP ===
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
maxHeap.offer(5); maxHeap.offer(3); maxHeap.offer(8);
System.out.println(maxHeap.peek()); // 8 – maximum element

```

```

// === CUSTOM COMPARATOR – min-heap by array's first element ===
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
pq.offer(new int[]{3, "some data".length()});
pq.offer(new int[]{1, "more".length()});
pq.offer(new int[]{2, "data".length()});
System.out.println(Arrays.toString(pq.poll())); // [1, 4]

```

```

// === DSA PATTERNS ===

```

```

// Pattern 1: Kth Largest Element
int[] nums = {3, 2, 1, 5, 6, 4};
int k = 2;
PriorityQueue<Integer> kthLargest = new PriorityQueue<>(); // min-heap
for (int n : nums) {
 kthLargest.offer(n);
 if (kthLargest.size() > k) kthLargest.poll(); // keep only k largest
}
System.out.println(kthLargest.peek()); // 5 – kth largest

```

```

// Pattern 2: Merge K Sorted Arrays (using heap)
// Pattern 3: Dijkstra's shortest path (min-heap on distances)

```

```

// === IMPORTANT: PriorityQueue does NOT support O(1) contains or remove by value ===
// contains() is O(n), remove(Object) is O(n)
// For efficient decrease-key: use TreeSet or custom indexed heap

```

```

}
}
...

```

### ### Key Points

- `PriorityQueue` default: **min-heap** (smallest element at top)

- `offer(x)` → insert  $O(\log n)$  | `poll()` → remove minimum  $O(\log n)$  | `peek()` → look at minimum  $O(1)$
- `PriorityQueue` does NOT allow null
- Does NOT support  $O(1)$  random access — `contains()` is  $O(n)$
- Use `(a, b) -> b - a` comparator for max-heap with integers; `Collections.reverseOrder()` is cleaner

---

## ## 30. Sorting & Comparators

### ### Explanation

Java has built-in sort for arrays (`Arrays.sort()`) and collections (`Collections.sort()` or `.sort()`). For custom sorting, implement `Comparator` or `Comparable`.

### ### Examples

```
```java
```

```
import java.util.*;
```

```
public class SortingDemo {
```

```
    // === COMPARABLE — natural ordering (define once in the class) ===
```

```
    static class Student implements Comparable<Student> {
```

```
        String name;
```

```
        int grade;
```

```
        Student(String name, int grade) {
```

```
            this.name = name;
```

```
            this.grade = grade;
```

```
        }
```

```
        @Override
```

```
        public int compareTo(Student other) {
```

```
            return this.grade - other.grade; // ascending by grade
```

```
            // return other.grade - this.grade; // descending
```

```
            // return this.name.compareTo(other.name); // by name
```

```
        }
```

```
        @Override
```

```
        public String toString() { return name + "(" + grade + ")"; }
```

```
    }
```

```
    public static void main(String[] args) {
```

```
// === PRIMITIVE ARRAY SORT ===
```

```
int[] arr = {5, 3, 8, 1, 9, 2};
```

```
Arrays.sort(arr);           // ascending: [1,2,3,5,8,9]
```

```
System.out.println(Arrays.toString(arr));
```

```
// No built-in descending for primitives — sort then reverse
```

```
// Or convert to Integer[] and use comparator
```

```
// === SORT PART OF ARRAY ===
```

```
int[] arr2 = {5, 3, 8, 1, 9, 2};
```

```
Arrays.sort(arr2, 1, 4); // sort indices 1-3 → [5, 1, 3, 8, 9, 2]
```

```
// === STRING SORT ===
```

```
String[] words = {"banana", "apple", "cherry", "date"};
```

```
Arrays.sort(words);           // alphabetical
```

```
Arrays.sort(words, (a, b) -> b.compareTo(a)); // reverse alphabetical
```

```
Arrays.sort(words, Comparator.comparingInt(String::length)); // by length
```

```
// === OBJECT ARRAY SORT — using Comparable ===
```

```
Student[] students = {  
    new Student("Alice", 92),  
    new Student("Bob", 85),  
    new Student("Charlie", 98)  
};
```

```
Arrays.sort(students); // uses compareTo — ascending by grade
```

```
System.out.println(Arrays.toString(students)); // [Bob(85), Alice(92), Charlie(98)]
```

```
// === OBJECT ARRAY SORT — using Comparator (without changing class) ===
```

```
Arrays.sort(students, (a, b) -> a.name.compareTo(b.name)); // by name
```

```
Arrays.sort(students, Comparator.comparing(s -> s.name)); // same, cleaner
```

```
Arrays.sort(students, Comparator.comparingInt((Student s) -> s.grade).reversed());
```

```
// Chained comparators:
```

```
Arrays.sort(students, Comparator  
    .comparingInt((Student s) -> s.grade)  
    .thenComparing(s -> s.name)); // sort by grade, then by name for ties
```

```
// === LIST SORT ===
```

```
List<Integer> list = new ArrayList<>(Arrays.asList(5, 3, 8, 1, 9));
```

```
Collections.sort(list);           // ascending
```

```
list.sort(Collections.reverseOrder()); // descending
```

```
list.sort((a, b) -> a - b);       // ascending lambda
```

```
list.sort((a, b) -> b - a); // descending lambda
```

```
// === COMPARATOR: compareTo contract ===
```

```
// compareTo returns:
```

```
// negative → this < other
```

```
// 0 → this == other
```

```
// positive → this > other
```

```
// For int: just return a - b for ascending (BUT watch for overflow with large negatives!)
```

```
// Safe: Integer.compare(a, b) instead of a - b
```

```
// === 2D ARRAY SORT (common in DSA) ===
```

```
int[][] intervals = {{3, 5}, {1, 4}, {2, 7}};
```

```
Arrays.sort(intervals, (a, b) -> a[0] - b[0]); // sort by start time
```

```
System.out.println(Arrays.deepToString(intervals)); // [[1,4],[2,7],[3,5]]
```

```
    }  
}  
...
```

Key Points

- **Comparable**: implement `compareTo` in the class — defines "natural" ordering
- **Comparator**: external comparison — pass as lambda or anonymous class — more flexible
- Comparator returning `a - b` sorts ascending; `b - a` sorts descending
- Use `Integer.compare(a, b)` instead of `a - b` to avoid integer overflow bugs in comparators
- `Arrays.sort` for primitives uses a variant of quicksort ($O(n \log n)$); for objects uses TimSort

Common Mistakes

```
```java
```

```
// Mistake 1 — integer overflow in comparator
```

```
Arrays.sort(arr, (a, b) -> a - b); // ❌ if a = Integer.MIN_VALUE and b = 1, overflows!
```

```
Arrays.sort(arr, (a, b) -> Integer.compare(a, b)); // ✅ safe
```

```
// Mistake 2 — forgetting sort is in-place for arrays
```

```
int[] original = {3, 1, 2};
```

```
int[] copy = original; // ❌ not a copy — alias!
```

```
Arrays.sort(copy); // modifies original too!
```

```
int[] copy2 = Arrays.copyOf(original, original.length); // ✅ true copy
```

```
...
```

```

```

## ## 31. Recursion

### ### Explanation

Recursion in Java works the same as in JavaScript — a function calls itself. Java has a default stack size of ~500-1000 recursive calls before `StackOverflowError`. For deep recursion, convert to iterative with an explicit stack.

### ### Examples

```
```java
```

```
public class RecursionDemo {
```

```
    // === BASIC RECURSION ===
```

```
    public static int factorial(int n) {  
        if (n <= 1) return 1;    // base case  
        return n * factorial(n - 1); // recursive case  
    }
```

```
    // === FIBONACCI (naive — exponential) ===
```

```
    public static int fib(int n) {  
        if (n <= 1) return n;  
        return fib(n - 1) + fib(n - 2);  
    }
```

```
    // === FIBONACCI with memoization ===
```

```
    static int[] memo = new int[100];  
    static { Arrays.fill(memo, -1); }
```

```
    public static int fibMemo(int n) {  
        if (n <= 1) return n;  
        if (memo[n] != -1) return memo[n];  
        return memo[n] = fibMemo(n - 1) + fibMemo(n - 2);  
    }
```

```
    // === BINARY SEARCH (recursive) ===
```

```
    public static int binarySearch(int[] arr, int target, int lo, int hi) {  
        if (lo > hi) return -1;    // base case — not found  
        int mid = lo + (hi - lo) / 2; // safe midpoint  
        if (arr[mid] == target) return mid;  
        if (arr[mid] < target) return binarySearch(arr, target, mid + 1, hi);  
        return binarySearch(arr, target, lo, mid - 1);  
    }
```

```

// === TREE RECURSION: max depth ===
// (Using a minimal TreeNode for illustration)
static class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public static int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}

// === BACKTRACKING: all subsets ===
public static List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

private static void backtrack(int[] nums, int start, List<Integer> current, List<List<Integer>> result) {
    result.add(new ArrayList<>(current)); // add current state
    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);           // choose
        backtrack(nums, i + 1, current, result); // explore
        current.remove(current.size() - 1); // unchoose
    }
}

public static void main(String[] args) {
    System.out.println(factorial(5)); // 120
    System.out.println(fib(10));      // 55 (slow)
    System.out.println(fibMemo(10));  // 55 (fast)

    int[] sorted = {1, 3, 5, 7, 9, 11};
    System.out.println(binarySearch(sorted, 7, 0, sorted.length - 1)); // 3

    System.out.println(subsets(new int[]{1, 2, 3}));
    // [], [1], [1,2], [1,2,3], [1,3], [2], [2,3], [3]
}
}
...

```

32. Lambda Expressions & Functional Interfaces

Explanation

Lambda expressions (Java 8+) are anonymous functions — a compact way to pass behavior as a parameter. They implement **functional interfaces** (interfaces with exactly one abstract method).

Examples

```
``java
import java.util.*;
import java.util.function.*;

public class LambdaDemo {

    // Functional interface — one abstract method
    @FunctionalInterface
    interface MathOperation {
        int operate(int a, int b);
    }

    public static void main(String[] args) {

        // === BASIC LAMBDA ===
        // (parameters) -> expression
        // (parameters) -> { statements; return value; }

        MathOperation add = (a, b) -> a + b;
        MathOperation multiply = (a, b) -> a * b;
        MathOperation max = (a, b) -> a > b ? a : b;

        System.out.println(add.operate(5, 3));    // 8
        System.out.println(multiply.operate(5, 3)); // 15

        // === BUILT-IN FUNCTIONAL INTERFACES ===
        // Function<T, R> — takes T, returns R → T → R
        Function<String, Integer> strLen = s -> s.length();
        System.out.println(strLen.apply("hello")); // 5

        // Predicate<T> — takes T, returns boolean → T → boolean
        Predicate<Integer> isEven = n -> n % 2 == 0;
```

```
System.out.println(isEven.test(4)); // true
System.out.println(isEven.test(3)); // false
```

```
// Consumer<T>    – takes T, returns nothing → T → void
Consumer<String> printer = s -> System.out.println(">> " + s);
printer.accept("hello"); // >> hello
```

```
// Supplier<T>    – takes nothing, returns T → () → T
Supplier<List<Integer>> listMaker = () -> new ArrayList<>();
List<Integer> newList = listMaker.get();
```

```
// BiFunction<T, U, R> – takes T and U, returns R
BiFunction<String, Integer, String> repeat = (s, n) -> s.repeat(n);
System.out.println(repeat.apply("ha", 3)); // hahaha
```

```
// === METHOD REFERENCES (shorthand for lambdas that just call a method) ===
// ClassName::methodName
```

```
List<String> names = Arrays.asList("Charlie", "Alice", "Bob");
```

```
names.sort(String::compareTo);    // same as (a, b) -> a.compareTo(b)
names.forEach(System.out::println); // same as s -> System.out.println(s)
```

```
// Static method reference:
Function<String, Integer> parser = Integer::parseInt; // String::method
System.out.println(parser.apply("42")); // 42
```

```
// Constructor reference:
Supplier<ArrayList<Integer>> listSupplier = ArrayList::new;
List<Integer> newList2 = listSupplier.get();
```

```
// === CHAINING FUNCTIONS ===
Function<Integer, Integer> doubleIt = n -> n * 2;
Function<Integer, Integer> addTen = n -> n + 10;

Function<Integer, Integer> doubleThenAdd = doubleIt.andThen(addTen);
System.out.println(doubleThenAdd.apply(5)); // 20 (5*2 + 10)
```

```
Predicate<Integer> isPositive = n -> n > 0;
Predicate<Integer> isSmall = n -> n < 100;
Predicate<Integer> isPositiveAndSmall = isPositive.and(isSmall);
```

```
}
```

```
}
```

33. Streams API

Explanation

The Streams API (Java 8+) lets you process collections with a functional, declarative style – similar to JavaScript's `map`, `filter`, `reduce`. Streams are lazy – they don't process data until a terminal operation is called.

Examples

```
```java
import java.util.*;
import java.util.stream.*;

public class StreamsDemo {
 public static void main(String[] args) {

 List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

 // === INTERMEDIATE OPERATIONS (lazy – build pipeline) ===
 // filter, map, flatMap, sorted, distinct, limit, skip, peek

 // filter – keep elements matching predicate
 List<Integer> evens = numbers.stream()
 .filter(n -> n % 2 == 0)
 .collect(Collectors.toList());
 System.out.println(evens); // [2, 4, 6, 8, 10]

 // map – transform each element
 List<Integer> doubled = numbers.stream()
 .map(n -> n * 2)
 .collect(Collectors.toList());
 System.out.println(doubled); // [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

 // filter + map chain:
 List<String> result = numbers.stream()
 .filter(n -> n % 2 == 0)
 .map(n -> "num:" + n)
 .collect(Collectors.toList());
 }
}
```

```
System.out.println(result); // [num:2, num:4, ...]
```

```
// === TERMINAL OPERATIONS (trigger processing) ===
```

```
// collect, forEach, count, sum, min, max, reduce, findFirst, anyMatch, allMatch
```

```
long count = numbers.stream().filter(n -> n > 5).count(); // 5
```

```
int sum = numbers.stream().mapToInt(Integer::intValue).sum(); // 55
```

```
OptionalInt max = numbers.stream().mapToInt(Integer::intValue).max();
```

```
System.out.println(max.getAsInt()); // 10
```

```
// reduce – aggregate
```

```
int product = numbers.stream().reduce(1, (a, b) -> a * b); // 1*2*3...*10 = 3628800
```

```
// collect to different types:
```

```
Set<Integer> set = numbers.stream().collect(Collectors.toSet());
```

```
String joined = Stream.of("a", "b", "c").collect(Collectors.joining(", ")); // "a, b, c"
```

```
Map<Boolean, List<Integer>> partitioned = numbers.stream()
```

```
 .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

```
// {false=[1,3,5,7,9], true=[2,4,6,8,10]}
```

```
// groupingBy
```

```
List<String> words = Arrays.asList("hello", "world", "java", "hi", "hey");
```

```
Map<Integer, List<String>> byLength = words.stream()
```

```
 .collect(Collectors.groupingBy(String::length));
```

```
System.out.println(byLength); // {2=[hi], 3=[hey], 4=[java], 5=[hello, world]}
```

```
// sorted
```

```
List<Integer> sorted = numbers.stream()
```

```
 .sorted(Comparator.reverseOrder())
```

```
 .collect(Collectors.toList()); // [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

```
// distinct
```

```
List<Integer> withDups = Arrays.asList(1, 2, 2, 3, 3, 3);
```

```
List<Integer> unique = withDups.stream().distinct().collect(Collectors.toList()); // [1,2,3]
```

```
// flatMap – flatten nested collections
```

```
List<List<Integer>> nested = Arrays.asList(Arrays.asList(1, 2), Arrays.asList(3, 4), Arrays.asList(5));
```

```
List<Integer> flat = nested.stream()
```

```
 .flatMap(Collection::stream)
```

```
 .collect(Collectors.toList()); // [1, 2, 3, 4, 5]
```

```
// anyMatch, allMatch, noneMatch
```

```

boolean anyNeg = numbers.stream().anyMatch(n -> n < 0); // false
boolean allPos = numbers.stream().allMatch(n -> n > 0); // true
boolean noneNeg = numbers.stream().noneMatch(n -> n < 0); // true

// findFirst
Optional<Integer> firstEven = numbers.stream().filter(n -> n % 2 == 0).findFirst();
firstEven.ifPresent(n -> System.out.println("First even: " + n)); // 2
}
}
...

```

### ### Key Points

- Streams are **one-time use** — once consumed, create a new stream
- **Lazy evaluation**: intermediate operations only run when a terminal operation is called
- For primitive streams (avoid boxing overhead): `IntStream`, `LongStream`, `DoubleStream`
- `collect(Collectors.toList())` → Java 16+: `.toList()` (shorter, immutable)

---

## ## 34. StringBuilder

### ### Explanation

`StringBuilder` is a **mutable string buffer** — use it when building strings incrementally (e.g., in loops). Much faster than `String` concatenation in loops because it doesn't create a new object every time.

### ### Examples

```

```java
public class StringBuilderDemo {
    public static void main(String[] args) {

        // === CREATING ===
        StringBuilder sb = new StringBuilder();      // empty
        StringBuilder sb2 = new StringBuilder("Hello"); // from string
        StringBuilder sb3 = new StringBuilder(100);  // with initial capacity

        // === BUILDING ===
        sb.append("Hello");    // append string
        sb.append(", ");
        sb.append("World");
        sb.append("!");
    }
}

```

```
sb.append(42);          // can append any type
sb.append(true);
sb.append(' ');        // can append char
System.out.println(sb.toString()); // "Hello, World!42true "
```

```
// Chaining (each method returns 'this'):
StringBuilder built = new StringBuilder()
    .append("Java")
    .append(" is")
    .append(" great!");
System.out.println(built); // "Java is great!"
```

```
// === MODIFICATION ===
sb2.insert(5, " Beautiful"); // insert at index 5 → "Hello Beautiful"
sb2.delete(5, 15);          // delete index 5 to 14 → "Hello"
sb2.replace(0, 5, "Hi");    // replace range → "Hi"
sb2.reverse();              // reverse → "iH"
```

```
// === ACCESS ===
StringBuilder s = new StringBuilder("Hello");
System.out.println(s.length()); // 5
System.out.println(s.charAt(0)); // 'H'
System.out.println(s.indexOf("l")); // 2
s.setCharAt(0, 'h');           // mutate character at index
System.out.println(s);         // "hello"
```

```
// === TO STRING ===
String result = sb.toString();
```

```
// === DSA: building result strings ===
```

```
// Example 1: reverse words in a sentence
String sentence = "Hello World Java";
String[] words = sentence.split(" ");
StringBuilder reversed = new StringBuilder();
for (int i = words.length - 1; i >= 0; i--) {
    reversed.append(words[i]);
    if (i > 0) reversed.append(" ");
}
System.out.println(reversed); // "Java World Hello"
```

```
// Example 2: counting in a loop (faster than +=)
StringBuilder digits = new StringBuilder();
```

```

for (int i = 0; i < 100; i++) {
    digits.append(i).append(",");
}
System.out.println(digits.toString()); // "0,1,2,...,99,"

// Example 3: palindrome check using StringBuilder.reverse()
String word = "racecar";
boolean isPalindrome = word.equals(new StringBuilder(word).reverse().toString());
System.out.println(isPalindrome); // true
}
}
...

```

Key Points

- `StringBuilder` is NOT thread-safe — for thread safety, use `StringBuffer` (slower)
- For DSA, always use `StringBuilder` instead of `String` concatenation in loops
- `sb.reverse()` is an $O(n)$ in-place reverse
- `sb.charAt(i)` and `sb.setCharAt(i, c)` give mutable character access — more powerful than String

Part 2 — Java for DSA

Input/Output for Competitive Programming / LeetCode

Explanation

LeetCode provides class and method skeletons — you just fill in the method body. For competitive programming, you may need fast I/O.

Examples

```

```java
import java.util.*;
import java.io.*;

public class IOPatterns {

 // === LEETCODE STYLE (most common — method given, you fill it in) ===
 // Example: Two Sum problem

```

```

public int[] twoSum(int[] nums, int target) {
 Map<Integer, Integer> map = new HashMap<>();
 for (int i = 0; i < nums.length; i++) {
 int complement = target - nums[i];
 if (map.containsKey(complement)) {
 return new int[]{map.get(complement), i};
 }
 map.put(nums[i], i);
 }
 return new int[]{};
}

```

// === SCANNER (standard input) ===

```

public static void scannerDemo() {
 Scanner sc = new Scanner(System.in);

 int n = sc.nextInt(); // read int
 long l = sc.nextLong(); // read long
 double d = sc.nextDouble(); // read double
 String s = sc.next(); // read word (stops at whitespace)
 String line = sc.nextLine(); // read full line

 // Read array:
 int[] arr = new int[n];
 for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

 sc.close();
}

```

// === FAST I/O (BufferedReader – much faster for large inputs) ===

```

public static void fastIO() throws IOException {
 BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

 int n = Integer.parseInt(br.readLine().trim());
 StringTokenizer st = new StringTokenizer(br.readLine());

 int[] arr = new int[n];
 for (int i = 0; i < n; i++) {
 arr[i] = Integer.parseInt(st.nextToken());
 }

 // Fast output
 StringBuilder out = new StringBuilder();

```

```

 for (int x : arr) out.append(x).append("\n");
 System.out.print(out);
 }
}
...

```

## ## Arrays (DSA Focus)

### ### 1. Explanation

Java arrays and ArrayLists are the foundation of most DSA problems. Master initialization patterns, two-pointer, and sliding window.

### ### 2. Example

```

```java
import java.util.*;

public class ArrayDSA {

    // === INITIALIZATION PATTERNS ===

    // Fixed-size array with default values:
    int[] dp = new int[n];          // all zeros
    boolean[] visited = new boolean[n]; // all false
    int[] memo = new int[n];
    Arrays.fill(memo, -1);          // fill with -1
    Arrays.fill(dp, Integer.MAX_VALUE); // fill with max

    // 2D array:
    int[][] grid = new int[rows][cols];
    boolean[][] seen = new boolean[rows][cols];

    // === TWO POINTER PATTERN ===
    public int[] twoSumSorted(int[] nums, int target) {
        int l = 0, r = nums.length - 1;
        while (l < r) {
            int sum = nums[l] + nums[r];
            if (sum == target) return new int[]{l, r};
            else if (sum < target) l++;
            else r--;
        }
    }
}

```

```
    return new int[]{};
}
```

```
// === SLIDING WINDOW (fixed size) ===
```

```
public double maxAverage(int[] nums, int k) {
    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += nums[i];
    int maxSum = windowSum;
    for (int i = k; i < nums.length; i++) {
        windowSum += nums[i] - nums[i - k]; // slide: add new, remove old
        maxSum = Math.max(maxSum, windowSum);
    }
    return (double) maxSum / k;
}
```

```
// === SLIDING WINDOW (variable size) ===
```

```
public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> map = new HashMap<>();
    int max = 0;
    int left = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (map.containsKey(c) && map.get(c) >= left) {
            left = map.get(c) + 1; // shrink window
        }
        map.put(c, right);
        max = Math.max(max, right - left + 1);
    }
    return max;
}
```

```
// === PREFIX SUM ===
```

```
public int[] buildPrefixSum(int[] nums) {
    int[] prefix = new int[nums.length + 1];
    for (int i = 0; i < nums.length; i++) {
        prefix[i + 1] = prefix[i] + nums[i];
    }
    return prefix;
}
```

```
// Sum from index l to r (inclusive): prefix[r+1] - prefix[l]
```

```
// === KADANE'S ALGORITHM (maximum subarray sum) ===
```

```
public int maxSubArray(int[] nums) {
```

```

    int currentSum = nums[0], maxSum = nums[0];
    for (int i = 1; i < nums.length; i++) {
        currentSum = Math.max(nums[i], currentSum + nums[i]);
        maxSum = Math.max(maxSum, currentSum);
    }
    return maxSum;
}
}
...

```

3. Is this important for DSA?

****YES — foundational****

4. How it is used in DSA

- Two-pointer: sorted array problems, trapping water, palindrome checking
- Sliding window: substring problems, maximum/minimum in window
- Prefix sum: range sum queries, subarray sum equals k

Sorting (DSA Focus)

1. Explanation

Sorting is a prerequisite for binary search, greedy algorithms, interval problems, and two-pointer on sorted arrays.

2. Example

```
```java
```

```
import java.util.*;
```

```
public class SortingDSA {
```

```
 // Sort primitives
```

```
 int[] arr = {5, 3, 8, 1};
```

```
 Arrays.sort(arr); // ascending
```

```
 // Sort object array by custom key
```

```
 int[][] intervals = {{3,5},{1,4},{2,7}};
```

```
 Arrays.sort(intervals, (a, b) -> a[0] - b[0]); // by start time
```

```
 // Sort List
```

```
 List<Integer> list = new ArrayList<>(Arrays.asList(3, 1, 4, 1, 5));
```

```

Collections.sort(list); // ascending
list.sort((a, b) -> b - a); // descending

// Sort String characters
String s = "dcba";
char[] chars = s.toCharArray();
Arrays.sort(chars);
String sorted = new String(chars); // "abcd"

// Anagram check using sorting:
boolean isAnagram(String s1, String s2) {
 if (s1.length() != s2.length()) return false;
 char[] a = s1.toCharArray(), b = s2.toCharArray();
 Arrays.sort(a); Arrays.sort(b);
 return Arrays.equals(a, b);
}

// Binary search (on sorted array):
int idx = Arrays.binarySearch(arr, target); // returns index, or negative if not found
// Manual binary search:
int lo = 0, hi = arr.length - 1;
while (lo <= hi) {
 int mid = lo + (hi - lo) / 2;
 if (arr[mid] == target) return mid;
 else if (arr[mid] < target) lo = mid + 1;
 else hi = mid - 1;
}
}
...

```

### 3. Is this important for DSA?

**\*\*YES\*\***

### 4. How it is used in DSA

- Interval problems — sort by start
- Greedy algorithms — sort to enable local optimal choices
- Two-pointer on sorted arrays
- Binary search (requires sorted input)

---

## HashMap Patterns (DSA Focus)

### ### 1. Explanation

HashMap is the most commonly used data structure for optimizing brute-force  $O(n^2)$  solutions to  $O(n)$ .

### ### 2. Example

```
```java
```

```
import java.util.*;
```

```
public class HashMapDSA {
```

```
    // Pattern 1 — Frequency Counter
```

```
    boolean areAnagrams(String s1, String s2) {  
        if (s1.length() != s2.length()) return false;  
        int[] freq = new int[26];  
        for (char c : s1.toCharArray()) freq[c - 'a']++;  
        for (char c : s2.toCharArray()) {  
            if (--freq[c - 'a'] < 0) return false;  
        }  
        return true;  
    }  
}
```

```
    // Pattern 2 — Two Sum (complement lookup)
```

```
    int[] twoSum(int[] nums, int target) {  
        Map<Integer, Integer> seen = new HashMap<>();  
        for (int i = 0; i < nums.length; i++) {  
            int complement = target - nums[i];  
            if (seen.containsKey(complement)) {  
                return new int[]{seen.get(complement), i};  
            }  
            seen.put(nums[i], i);  
        }  
        return new int[]{};  
    }  
}
```

```
    // Pattern 3 — Group by key (Group Anagrams)
```

```
    List<List<String>> groupAnagrams(String[] strs) {  
        Map<String, List<String>> map = new HashMap<>();  
        for (String s : strs) {  
            char[] chars = s.toCharArray();  
            Arrays.sort(chars);  
            String key = new String(chars);  
            map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);  
        }  
    }  
}
```

```
return new ArrayList<> (map.values());  
}
```

// Pattern 4 — Prefix sum with HashMap (subarray sum equals k)

```
int subarraySum(int[] nums, int k) {  
    Map<Integer, Integer> prefixCount = new HashMap<>();  
    prefixCount.put(0, 1);  
    int count = 0, sum = 0;  
    for (int n : nums) {  
        sum += n;  
        count += prefixCount.getOrDefault(sum - k, 0);  
        prefixCount.merge(sum, 1, Integer::sum);  
    }  
    return count;  
}
```

// Pattern 5 — Sliding Window with HashMap (minimum window substring)

```
String minWindow(String s, String t) {  
    Map<Character, Integer> need = new HashMap<>();  
    for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);  
  
    int have = 0, required = need.size();  
    int[] ans = {-1, 0, 0};  
    Map<Character, Integer> window = new HashMap<>();  
    int left = 0;  
  
    for (int right = 0; right < s.length(); right++) {  
        char c = s.charAt(right);  
        window.merge(c, 1, Integer::sum);  
        if (need.containsKey(c) && window.get(c).equals(need.get(c))) have++;  
  
        while (have == required) {  
            if (ans[0] == -1 || right - left + 1 < ans[0]) {  
                ans[0] = right - left + 1;  
                ans[1] = left;  
                ans[2] = right;  
            }  
            char lc = s.charAt(left++);  
            window.merge(lc, -1, Integer::sum);  
            if (need.containsKey(lc) && window.get(lc) < need.get(lc)) have--;  
        }  
    }  
  
    return ans[0] == -1 ? "" : s.substring(ans[1], ans[2] + 1);  
}
```

```
}  
}  
...
```

3. Is this important for DSA?

****YES – absolutely critical****

4. How it is used in DSA

- Reduces $O(n^2)$ brute force to $O(n)$
- Core to sliding window, two sum, anagram, and grouping problems
- Prefix sum with HashMap: subarray problems

Linked List Node (Custom Data Structure)

2. Example

```
```java
```

```
public class ListNodeDemo {
```

```
 // Node definition (given in LeetCode problems)
```

```
 static class ListNode {
```

```
 int val;
```

```
 ListNode next;
```

```
 ListNode() {}
```

```
 ListNode(int val) { this.val = val; }
```

```
 ListNode(int val, ListNode next) { this.val = val; this.next = next; }
```

```
 }
```

```
 // Build list from array
```

```
 static ListNode arrayToList(int[] arr) {
```

```
 ListNode dummy = new ListNode(0);
```

```
 ListNode curr = dummy;
```

```
 for (int val : arr) {
```

```
 curr.next = new ListNode(val);
```

```
 curr = curr.next;
```

```
 }
```

```
 return dummy.next;
```

```
 }
```

```
 // Reverse linked list
```

```
 static ListNode reverseList(ListNode head) {
```

```

ListNode prev = null, curr = head;
while (curr != null) {
 ListNode next = curr.next;
 curr.next = prev;
 prev = curr;
 curr = next;
}
return prev;
}

// Fast & slow pointer — detect cycle
static boolean hasCycle(ListNode head) {
 ListNode slow = head, fast = head;
 while (fast != null && fast.next != null) {
 slow = slow.next;
 fast = fast.next.next;
 if (slow == fast) return true;
 }
 return false;
}

// Merge two sorted lists
static ListNode mergeTwoLists(ListNode l1, ListNode l2) {
 ListNode dummy = new ListNode(0), curr = dummy;
 while (l1 != null && l2 != null) {
 if (l1.val <= l2.val) { curr.next = l1; l1 = l1.next; }
 else { curr.next = l2; l2 = l2.next; }
 curr = curr.next;
 }
 curr.next = (l1 != null) ? l1 : l2;
 return dummy.next;
}

// Find middle (slow & fast pointer)
static ListNode findMiddle(ListNode head) {
 ListNode slow = head, fast = head;
 while (fast != null && fast.next != null) {
 slow = slow.next;
 fast = fast.next.next;
 }
 return slow; // for even-length list, returns second middle
}
}

```

```

3. Is this important for DSA?

****YES****

4. How it is used in DSA

- Reversal, cycle detection, merging, finding middle
- Most linked list LeetCode problems use this exact pattern

Tree Node (Custom Data Structure)

2. Example

```java

import java.util.\*;

public class TreeNodeDemo {

    // Node definition (given in LeetCode)

    static class TreeNode {

        int val;

        TreeNode left, right;

        TreeNode() {}

        TreeNode(int val) { this.val = val; }

        TreeNode(int val, TreeNode left, TreeNode right) {

            this.val = val; this.left = left; this.right = right;

        }

    }

    // DFS — inorder traversal (left → root → right)

    static void inorder(TreeNode root, List<Integer> result) {

        if (root == null) return;

        inorder(root.left, result);

        result.add(root.val);

        inorder(root.right, result);

    }

    // DFS — max depth

    static int maxDepth(TreeNode root) {

        if (root == null) return 0;

        return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));

```

}

// BFS — level order traversal
static List<List<Integer>> levelOrder(TreeNode root) {
 List<List<Integer>> result = new ArrayList<>();
 if (root == null) return result;
 Queue<TreeNode> queue = new ArrayDeque<>();
 queue.offer(root);
 while (!queue.isEmpty()) {
 int size = queue.size();
 List<Integer> level = new ArrayList<>();
 for (int i = 0; i < size; i++) {
 TreeNode node = queue.poll();
 level.add(node.val);
 if (node.left != null) queue.offer(node.left);
 if (node.right != null) queue.offer(node.right);
 }
 result.add(level);
 }
 return result;
}

// Check if BST is valid
static boolean isValidBST(TreeNode root, long min, long max) {
 if (root == null) return true;
 if (root.val <= min || root.val >= max) return false;
 return isValidBST(root.left, min, root.val) &&
 isValidBST(root.right, root.val, max);
}
// Call: isValidBST(root, Long.MIN_VALUE, Long.MAX_VALUE)

// Lowest Common Ancestor
static TreeNode lca(TreeNode root, TreeNode p, TreeNode q) {
 if (root == null || root == p || root == q) return root;
 TreeNode left = lca(root.left, p, q);
 TreeNode right = lca(root.right, p, q);
 if (left != null && right != null) return root;
 return left != null ? left : right;
}
}
...

```

### 3. Is this important for DSA?

**\*\*YES\*\***

---

## ## Graph (Adjacency List & BFS/DFS)

### ### 2. Example

```
```java
```

```
import java.util.*;
```

```
public class GraphDSA {
```

```
    // === BFS TEMPLATE ===
```

```
    static int bfs(int start, int target, Map<Integer, List<Integer>> graph) {
```

```
        Queue<Integer> queue = new ArrayDeque<>();
```

```
        Set<Integer> visited = new HashSet<>();
```

```
        queue.offer(start);
```

```
        visited.add(start);
```

```
        int steps = 0;
```

```
        while (!queue.isEmpty()) {
```

```
            int size = queue.size();
```

```
            for (int i = 0; i < size; i++) {
```

```
                int curr = queue.poll();
```

```
                if (curr == target) return steps;
```

```
                for (int neighbor : graph.getOrDefault(curr, new ArrayList<>())) {
```

```
                    if (!visited.contains(neighbor)) {
```

```
                        visited.add(neighbor);
```

```
                        queue.offer(neighbor);
```

```
                    }
```

```
                }
```

```
            }
```

```
            steps++;
```

```
        }
```

```
        return -1;
```

```
    }
```

```
    // === DFS TEMPLATE (recursive) ===
```

```
    static void dfs(int node, Set<Integer> visited, Map<Integer, List<Integer>> graph) {
```

```
        visited.add(node);
```

```
        for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
```

```
            if (!visited.contains(neighbor)) {
```

```

        dfs(neighbor, visited, graph);
    }
}

// === DFS TEMPLATE (iterative with stack) ===
static void dfsIterative(int start, Map<Integer, List<Integer>> graph) {
    Deque<Integer> stack = new ArrayDeque<>();
    Set<Integer> visited = new HashSet<>();
    stack.push(start);
    while (!stack.isEmpty()) {
        int curr = stack.pop();
        if (visited.contains(curr)) continue;
        visited.add(curr);
        for (int neighbor : graph.getOrDefault(curr, new ArrayList<>())) {
            if (!visited.contains(neighbor)) stack.push(neighbor);
        }
    }
}

```

```

// === GRID BFS (very common in LeetCode) ===
static int[][] DIRS = {{0,1},{0,-1},{1,0},{-1,0}};

static int bfsGrid(char[][] grid, int startR, int startC) {
    int rows = grid.length, cols = grid[0].length;
    Queue<int[]> queue = new ArrayDeque<>();
    boolean[][] visited = new boolean[rows][cols];
    queue.offer(new int[]{startR, startC});
    visited[startR][startC] = true;
    int steps = 0;

    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            int[] curr = queue.poll();
            for (int[] dir : DIRS) {
                int nr = curr[0] + dir[0];
                int nc = curr[1] + dir[1];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols
                    && !visited[nr][nc] && grid[nr][nc] != '#') {
                    visited[nr][nc] = true;
                    queue.offer(new int[]{nr, nc});
                }
            }
        }
        steps++;
    }
    return steps;
}

```

```

    }
    }
    steps++;
}
return steps;
}

// === UNION-FIND (Disjoint Set Union) ===
static class UnionFind {
    int[] parent, rank;

    UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]); // path compression
        return parent[x];
    }

    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false; // already connected
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }

    boolean connected(int x, int y) { return find(x) == find(y); }
}
}
...

```

3. Is this important for DSA?

****YES — critical****

Dynamic Programming (DSA Focus)

2. Example

```
```java
import java.util.*;

public class DPDSA {

 // === MEMOIZATION TEMPLATE (top-down) ===
 Map<String, Integer> memo = new HashMap<>();

 int dp(int state1, int state2) {
 String key = state1 + "," + state2;
 if (memo.containsKey(key)) return memo.get(key);
 // base cases
 int result = 0; // compute recursively
 memo.put(key, result);
 return result;
 }

 // === TABULATION TEMPLATE (bottom-up) ===
 // Fibonacci with DP:
 int fib(int n) {
 if (n <= 1) return n;
 int[] dp = new int[n + 1];
 dp[1] = 1;
 for (int i = 2; i <= n; i++) dp[i] = dp[i-1] + dp[i-2];
 return dp[n];
 }

 // === 0/1 KNAPSACK ===
 int knapsack(int[] weights, int[] values, int capacity) {
 int n = weights.length;
 int[][] dp = new int[n + 1][capacity + 1];
 for (int i = 1; i <= n; i++) {
 for (int w = 0; w <= capacity; w++) {
 dp[i][w] = dp[i-1][w]; // don't take item i
 if (weights[i-1] <= w) {
 dp[i][w] = Math.max(dp[i][w], dp[i-1][w - weights[i-1]] + values[i-1]);
 }
 }
 }
 return dp[n][capacity];
 }
}
```

```
// === LONGEST COMMON SUBSEQUENCE ===
int lcs(String s1, String s2) {
 int m = s1.length(), n = s2.length();
 int[][] dp = new int[m + 1][n + 1];
 for (int i = 1; i <= m; i++) {
 for (int j = 1; j <= n; j++) {
 if (s1.charAt(i-1) == s2.charAt(j-1)) dp[i][j] = dp[i-1][j-1] + 1;
 else dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
 }
 }
 return dp[m][n];
}

// === COIN CHANGE ===
int coinChange(int[] coins, int amount) {
 int[] dp = new int[amount + 1];
 Arrays.fill(dp, amount + 1); // infinity
 dp[0] = 0;
 for (int i = 1; i <= amount; i++) {
 for (int coin : coins) {
 if (coin <= i) dp[i] = Math.min(dp[i], dp[i - coin] + 1);
 }
 }
 return dp[amount] > amount ? -1 : dp[amount];
}
...

```

### 3. Is this important for DSA?

**\*\*YES — core pattern for medium/hard problems\*\***

---

## Quick DSA Reference — Java Cheat Sheet

```
```java

```

```
// ——— ARRAY INITIALIZATION

```

```
int[] arr = new int[n];           // [0, 0, ..., 0]
Arrays.fill(arr, -1);            // [-1, -1, ..., -1]
int[][] grid = new int[rows][cols]; // 2D grid of zeros
boolean[] visited = new boolean[n]; // all false

```

// — COMMON MATH

```
Math.max(a, b)           // max of two values
Math.min(a, b)           // min of two values
Math.abs(n)              // absolute value
Math.pow(base, exp)      // base^exp (returns double)
Math.sqrt(n)             // square root (returns double)
Math.log(n)              // natural log
Integer.MAX_VALUE        // 2147483647 (~2.1 billion)
Integer.MIN_VALUE        // -2147483648
Long.MAX_VALUE           //  $9.2 \times 10^{18}$ 
```

// — INTEGER / LONG SAFE MID

```
int mid = lo + (hi - lo) / 2;    // safe midpoint (no overflow)
int mid = lo + ((hi - lo) >> 1); // same with bitwise
```

// — STRING TRICKS

```
char[] chars = s.toCharArray();    // String → char array
String back = new String(chars);    // char array → String
s.charAt(i) - 'a'                   // letter → 0-25 index
(char)('a' + index)                 // index → letter
new StringBuilder(s).reverse().toString() // reverse string
s.substring(l, r)                   // s[l..r-1]
String.valueOf(n)                   // int/char/etc → String
Integer.parseInt(s)                 // String → int
```

// — SORTING

```
Arrays.sort(arr)                  // ascending primitives
Arrays.sort(arr, (a, b) -> a[0] - b[0]) // sort 2D by first element
Collections.sort(list)            // ascending list
list.sort((a, b) -> b - a)        // descending list
Arrays.sort(objArr, Comparator.comparingInt(x -> x.key)) // by field
```

// — FREQUENCY COUNT

```
Map<Character, Integer> freq = new HashMap<>();
for (char c : s.toCharArray())
    freq.merge(c, 1, Integer::sum);
```

```
int[] freq = new int[26];           // for lowercase letters only
freq[c - 'a']++;
```

```
// ——— STACK (LIFO)
```

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(x);           // add top
stack.pop();             // remove top — O(1)
stack.peek();           // look at top — O(1)
stack.isEmpty();
```

```
// ——— QUEUE (FIFO)
```

```
Queue<Integer> queue = new ArrayDeque<>();
queue.offer(x);          // add to back — O(1)
queue.poll();            // remove from front — O(1)
queue.peek();            // look at front — O(1)
```

```
// ——— DEQUE (both ends)
```

```
Deque<Integer> dq = new ArrayDeque<>();
dq.offerFirst(x); dq.offerLast(x);
dq.pollFirst(); dq.pollLast();
dq.peekFirst(); dq.peekLast();
```

```
// ——— PRIORITY QUEUE (HEAP)
```

```
PriorityQueue<Integer> minPQ = new PriorityQueue<>();           // min-heap
PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Collections.reverseOrder()); // max-heap
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]); // by first element
minPQ.offer(x); minPQ.poll(); minPQ.peek();
```

```
// ——— TWO POINTERS
```

```
int l = 0, r = arr.length - 1;
while (l < r) { l++; r--; }
```

```
// ——— BINARY SEARCH
```

```
int lo = 0, hi = arr.length - 1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target) return mid;
```

```
    else if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
}
```

// ——— BFS TEMPLATE

```
Queue<Integer> queue = new ArrayDeque<>();
Set<Integer> visited = new HashSet<>();
queue.offer(start); visited.add(start);
while (!queue.isEmpty()) {
    int curr = queue.poll();
    for (int next : getNeighbors(curr)) {
        if (!visited.contains(next)) {
            visited.add(next);
            queue.offer(next);
        }
    }
}
```

// ——— DFS TEMPLATE (recursive)

```
void dfs(int node, Set<Integer> visited) {
    if (visited.contains(node)) return;
    visited.add(node);
    for (int next : getNeighbors(node)) dfs(next, visited);
}
```

// ——— BACKTRACKING TEMPLATE

```
void backtrack(int start, List<Integer> current) {
    if (/* done */) { result.add(new ArrayList<>(current)); return; }
    for (int i = start; i < choices.length; i++) {
        current.add(choices[i]);
        backtrack(i + 1, current);
        current.remove(current.size() - 1);
    }
}
```

// ——— MEMOIZATION TEMPLATE

```
Map<String, Integer> memo = new HashMap<>();
int dp(int a, int b) {
    String key = a + "," + b;
```

```

    if (memo.containsKey(key)) return memo.get(key);
    // base cases
    int result = /* recursive computation */;
    memo.put(key, result);
    return result;
}

// ——— GRAPH (adjacency list)

```

```

Map<Integer, List<Integer>> graph = new HashMap<>();
graph.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
graph.computeIfAbsent(v, k -> new ArrayList<>()).add(u); // undirected

```

```

// ——— UNION-FIND

```

```

int[] parent = new int[n];
for (int i = 0; i < n; i++) parent[i] = i;

int find(int x) {
    return parent[x] == x ? x : (parent[x] = find(parent[x]));
}

```

```

void union(int x, int y) {
    parent[find(x)] = find(y);
}

```

```

// ——— SORTED MAP OPERATIONS

```

```

TreeMap<Integer, Integer> tm = new TreeMap<>();
tm.floorKey(x); // largest key ≤ x
tm.ceilingKey(x); // smallest key ≥ x
tm.firstKey(); // minimum key
tm.lastKey(); // maximum key
...

```

End of Java Complete Guide + DSA Mapping